

Lab 3: Game Access Control on FPGA

Tru Nguyen

PSID: 1911032

ECE 6370

1. Introduction

This project is the design of a game that can be played by two players on the DE0-CV FPGA board. Each player has one set of four slide switches on the board corresponding to a 4-bit binary number. Player 1 switches are the four rightmost switches (SW3-SW0) which SW0 is the least significant bit. Player 2 switches are the four leftmost switches (SW9-SW6) which SW6 is the least significant bit. Player 1 and Player 2's numbers are displayed on the rightmost 7-segment display and the second to leftmost display in hexadecimal value respectively. The sum of the two players' numbers is displayed on the third 7-segment display from the right in hexadecimal value. The leftmost LED is lit if the sum of the two numbers equal 4'b1111 or its right-adjacent LED is lit if otherwise.

Players have their own LOAD button that will display their respective number on the switches on the 7-segment displays. Before any player can load their number, the system password must be entered using Player 2's set of switches. If the password is successfully entered, the rightmost LED is lit up indicating logged-in status or the left-adjacent LED is lit if a correct password has not been entered. Once logged in and the password LOAD button is pressed again, players will be logged out. They cannot load their numbers until logged in again. The RESET button will log players out and set the board back to default settings.

The rule of the game is that Player 1 sets their number using the switches then Player 2 needs to set a number (one-time only) that the sum of both numbers equals 4'b1111 without looking at Player 1's switches. If Player 2 gets the right number, they win the round. If Player 2 gets the wrong number, Player 1 wins the round. The players switch roles after every round as Player 2 now sets a number for Player 1 to match.

In Lab 3, the system is enhanced by replacing Player 2 with a hardware-based true Random Number Generator (RNG) that produces a 4-bit value (0–15). The RNG is implemented using a 4-bit counter driven by the 50 MHz clock. A dedicated button is used to generate the random number: when the player presses and holds the button, the counter increments continuously, and when the button is released, the current counter value is captured and displayed as the target number. This button is intentionally not debounced so that the duration of the press determines the randomness.

The password protection and load functionality from Lab 2 are preserved. Before the correct password is entered, pressing either the random number generation button or the player LOAD button will have no effect. This is achieved by routing the RNG button through the access controller, which forces the signal inactive until authentication is successful. Once the password is verified, the RNG and load operations function normally.

Additionally, a timer feature is introduced to control gameplay. The game operates with a default countdown of 99 seconds, displayed in base-10 format on the 7-segment displays. Before password verification, the timer displays "00." After successful login, the timer is initialized to 99 seconds but does not begin counting until the GAME button is pressed. During this time, the player attempts to complete as many rounds as possible by selecting a number that, when added to the generated random number, equals 4'b1111.

When the timer reaches zero, the game ends and the player is no longer able to generate new random numbers or load inputs. Pressing the GAME button after timeout reloads the timer to 99 seconds, and pressing it again restarts the countdown. The logout functionality from Lab 2 is disabled in this version since the same button is repurposed as the GAME control.

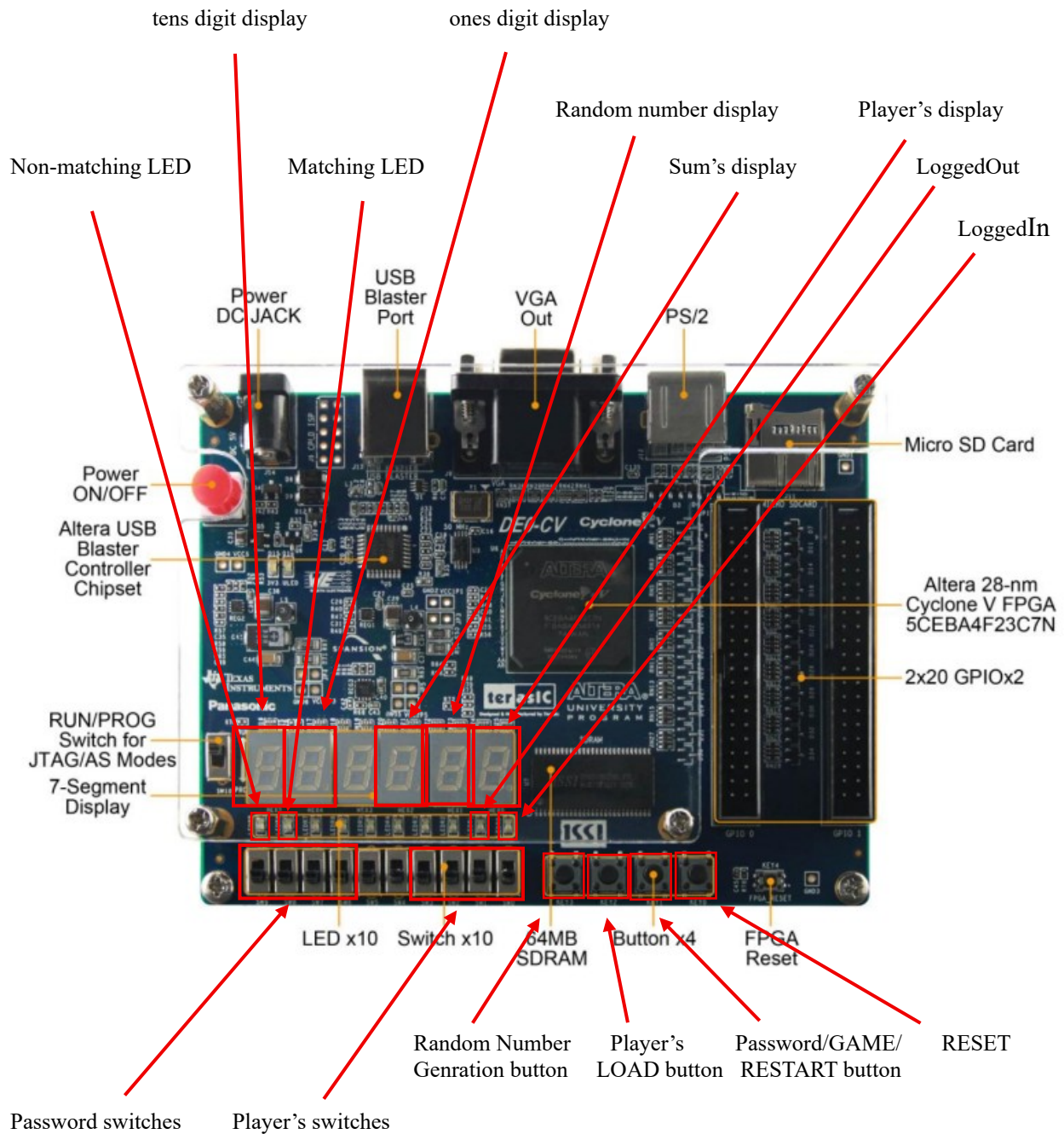


Figure 1: DE0-CV FPGA board with annotated Players' switches, displays, buttons and LEDs

2. System Architecture Design

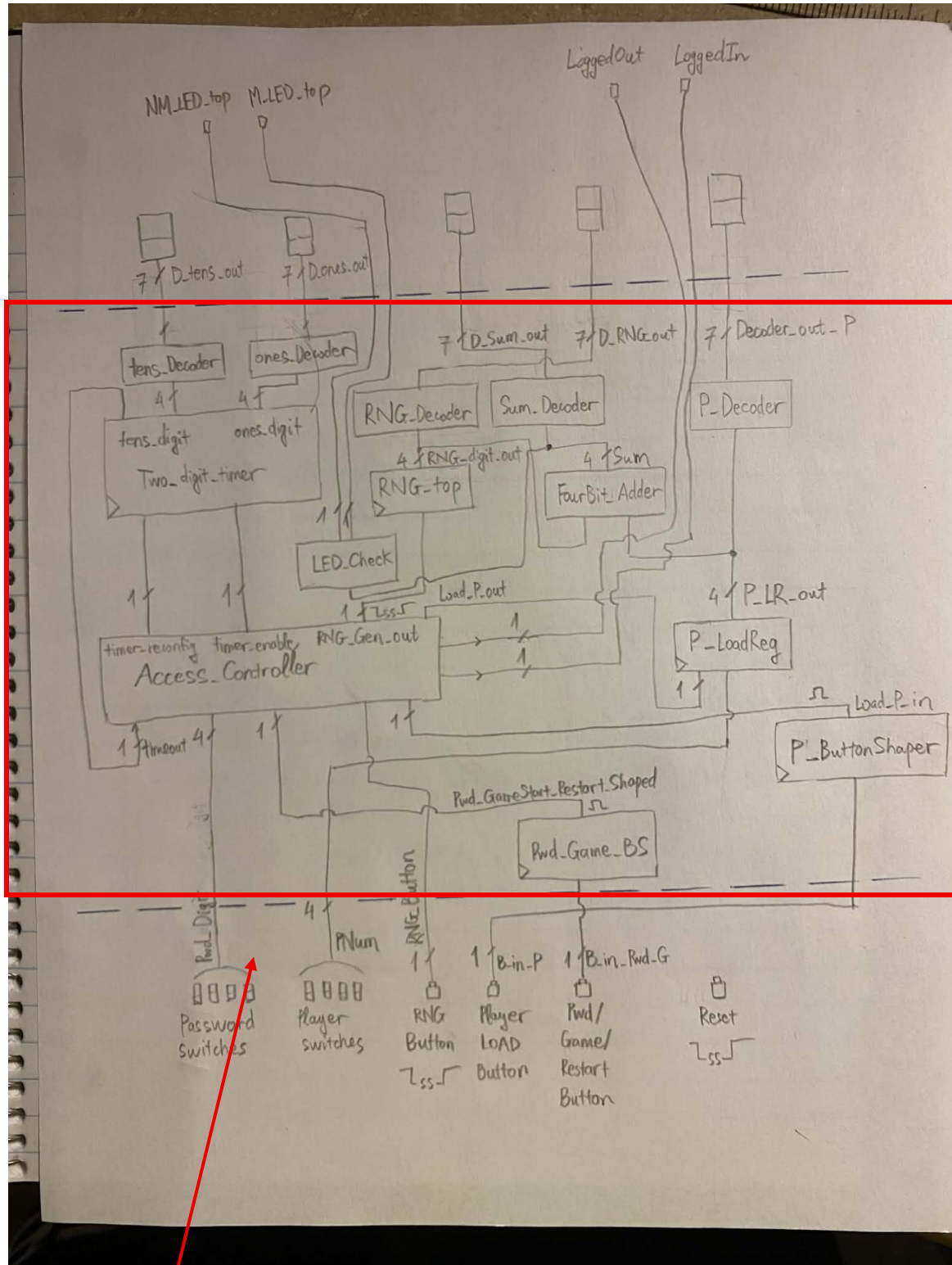
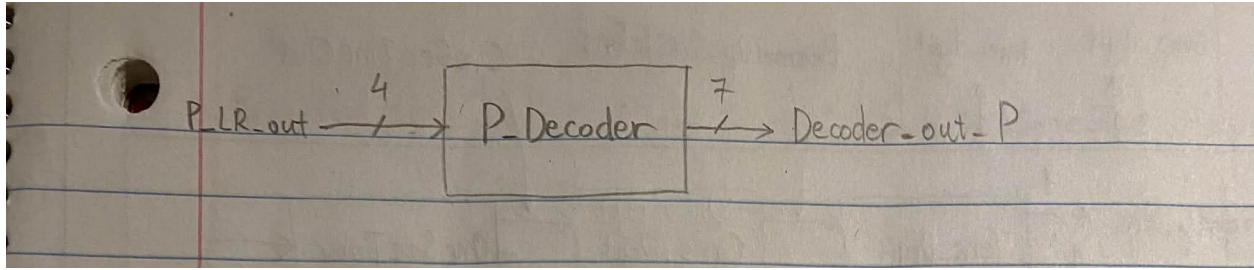


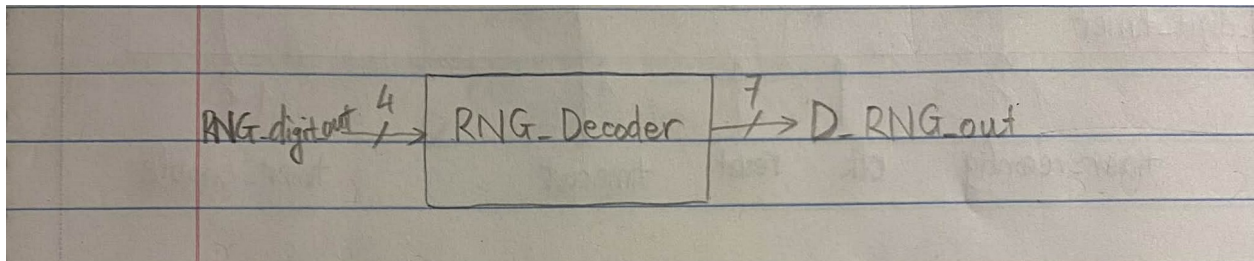
Figure 2: Top-level system architecture
Top-level module Lab2_NGUYEN_Tru

2.1. P_Decoder



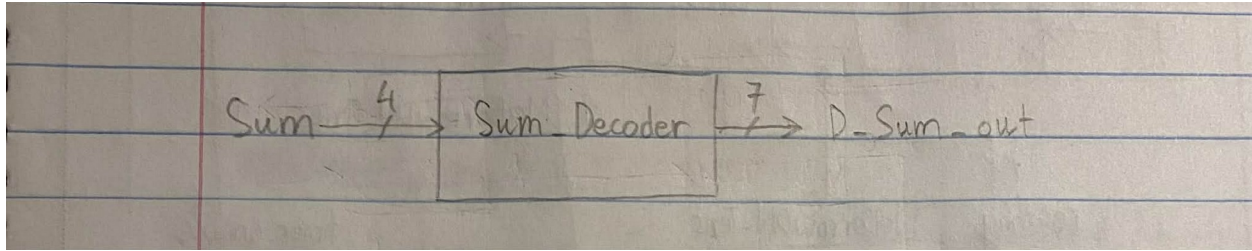
- **Function:** decodes the 4-bit binary number from Player's switches to respective hexadecimal value displayed on Player's display.
- **Input:**
[3:0] P_LR_out: Player's 4-bit binary number from switches.
- **Output:**
[6:0] Decoder_out_P: 7-bit value shows which indexed segment to be turned on and off on Player's display.

2.2. RNG_Decoder



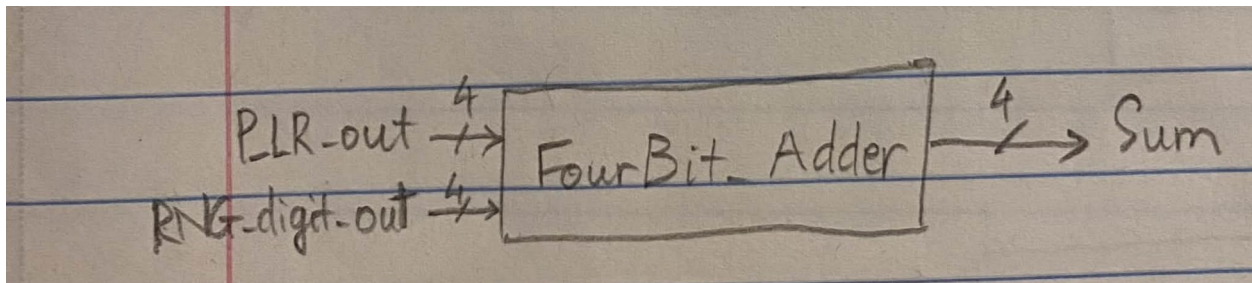
- **Function:** decodes the 4-bit binary number from the random number generator to respective hexadecimal value displayed on RNG's display.
- **Input:**
[3:0] RNG_digit_out: Randomly generated 4-bit binary number from RNG Button.
- **Output:**
[6:0] D_RNG_out_2: 7-bit value shows the randomly generated number on respective display.

2.3. Sum_Decoder



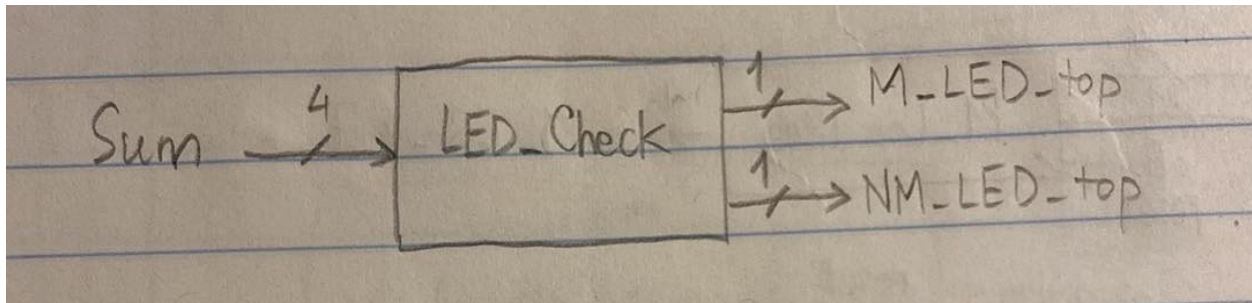
- **Function:** decodes the 4-bit output binary number from the FourBit_Adder module to respective hexadecimal value displayed on Sum display.
- **Input:**
[3:0] Sum: 4-bit binary sum of Player and the generated random 4-bit binary numbers.
- **Output:**
[6:0] D_Sum_out: 7-bit value shows which indexed segment to be turned on and off on Sum display.

2.4. FourBit_Adder



- **Function:** performs the sum of Player and the randomly generated s 4-bit binary numbers.
- **Input:**
[3:0] P_LR_out: Player's 4-bit binary number from switches held in P_LoadReg.
[3:0] RNG_digit_out: Randomly generated 4-bit binary random number.
- **Output:**
[3:0] Sum: holds the 4-bit binary sum of Player and the generated random 4-bit binary numbers.

2.5. LED_Check



- **Function:** verifies if the sum of Player 1 and 2's numbers equals 1111 (binary). If so, lights up the leftmost LED or the rightmost LED if otherwise.

- **Input:**

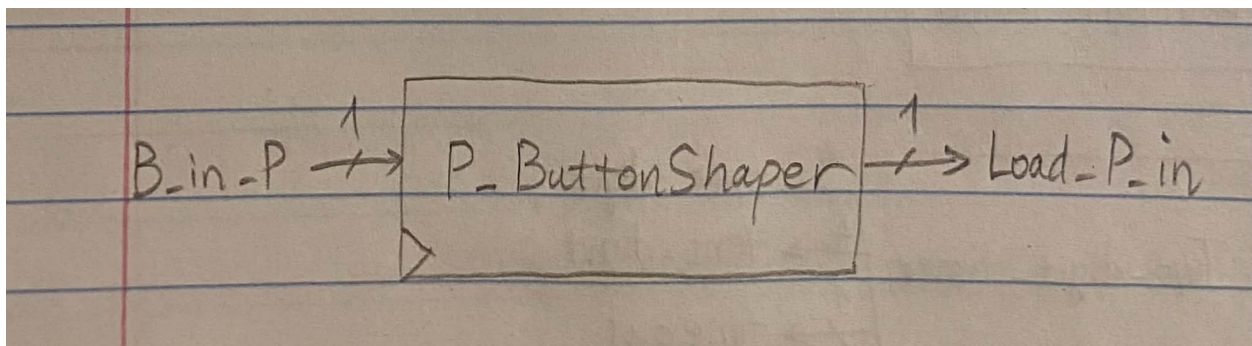
[3:0] Sum: holds the 4-bit binary sum of Player 1 and 2's 4-bit binary numbers.

- **Output:**

M_LED_top: drives the matching LED.

NM_LED_top: drives the non-matching LED.

2.6. P_ButtonShaper



- **Function:** Generates a single clock-cycle pulse when Player's button is pressed.

- **Input:**

B_in_P: raw Player's button push signal.

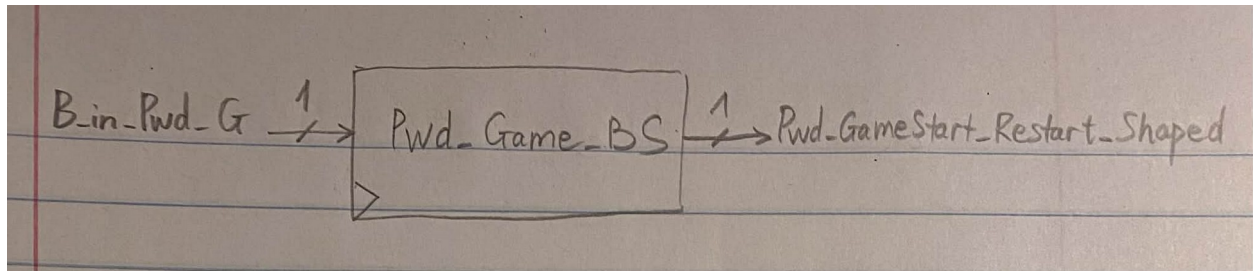
reset: active low reset.

clk: system clock.

- **Output:**

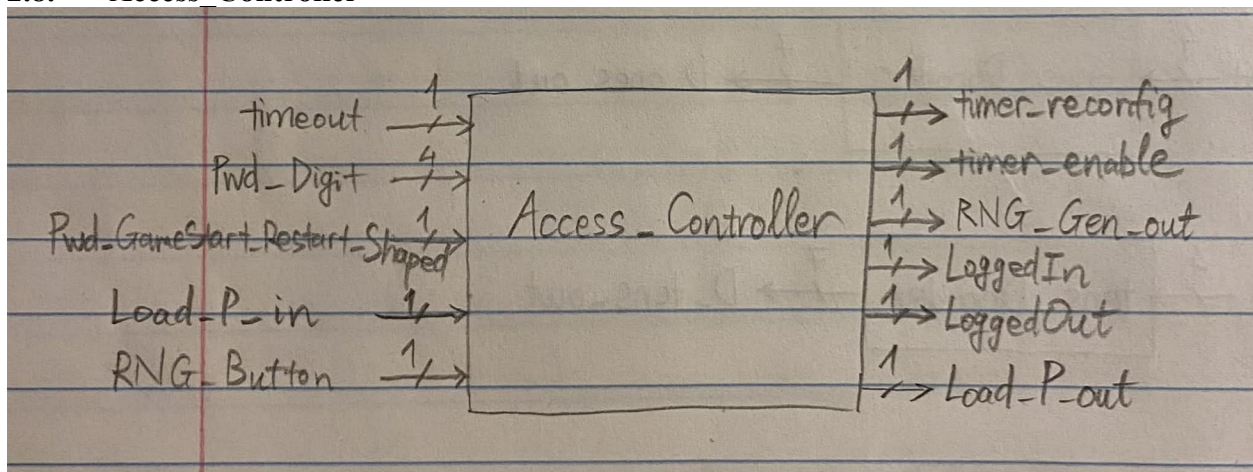
Load_P_in: Clean one-cycle pulse used to request loading Player's number.

2.7. Pwd_Game_BS



- **Function:** Generates a single clock-cycle pulse when password-enter button is pressed and to start or restart gameplay.
- **Input:**
 B_in_Pwd_G: raw password-enter/game start/game restart button push signal.
 reset: active low reset.
 clk: system clock.
- **Output:**
 Pwd_GameStart_Restart_Shaped: Clean one-cycle pulse sent to Access_Controller.

2.8. Access Controller



- **Function:** checks a 4-digit password (1032), controls login/logout, and enables loading of Player 1 and Player 2' numbers only when logged in.
- **Input:**
 Pwd_GameStart_Restart_Shaped: one-cycle pulse to enter a password digit and start or restart game.
 [3:0] Pwd_Digit: current password digit value.
 Load_P_in: Load request for Player's number.
 RNG_Button: raw button push signal from the random number generation button.
 timeout: stops timer counting after time is up.
 reset: active low reset.
 clk: system clock.
- **Output:**
 Load_P_out: Clean one-cycle pulse allowing Player's number to be loaded.

LoggedIn: LED indicator (user authenticated).

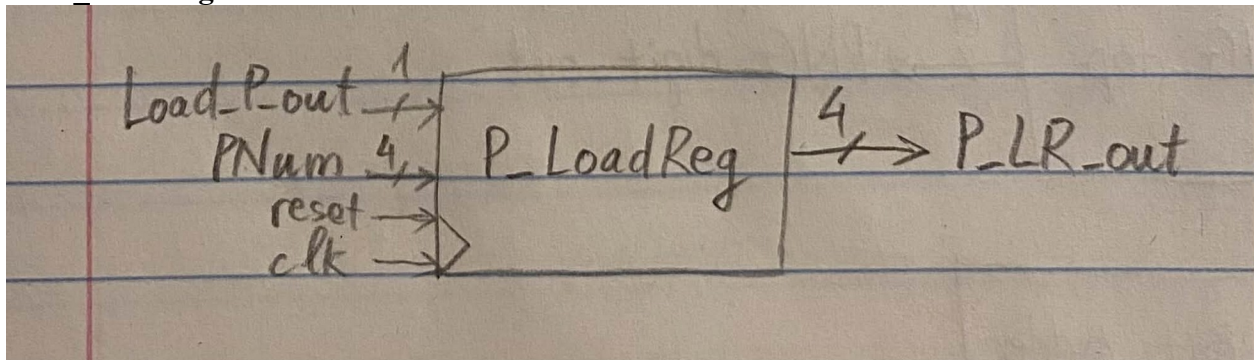
LoggedOut: LED indicator (user not authenticated).

timer_reconfig: reload digit 9 to both ones and tens display.

timer_enable: enable timer to start counting down.

RNG_Gen_out: raw button push signal from the random number generation button allowed to pass to the RNG_top module.

2.9. P_LoadReg



- **Function:** stores Player's 4-bit number when enabled.

- **Input:**

[3:0] PNum: Player's 4-bit input number.

Load_P_out: Load enable signal.

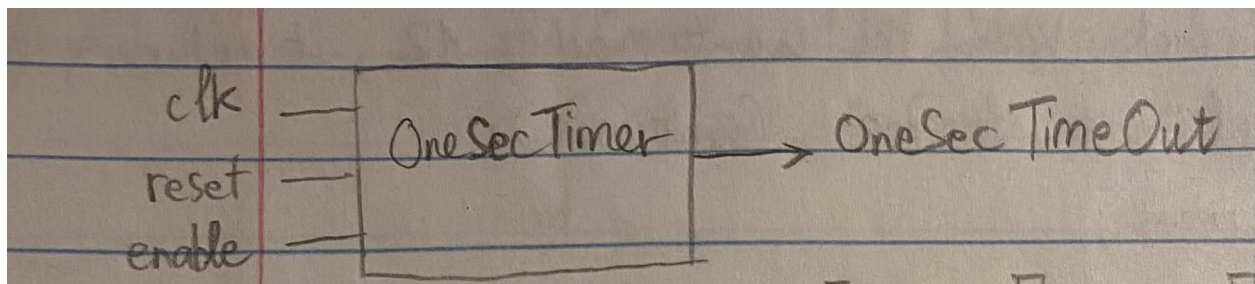
reset: active low reset.

clk: system clock.

- **Output:**

P_LR_out: Player's stored 4-bit value.

2.10. OneSecTimer



OneSecTimer module is the top-level timing module that generates a 1-second timeout signal by cascading three smaller timers. It uses a 1 ms timer, then counts those pulses to form 100 ms, and finally counts again to produce a 1-second pulse.

Inputs:

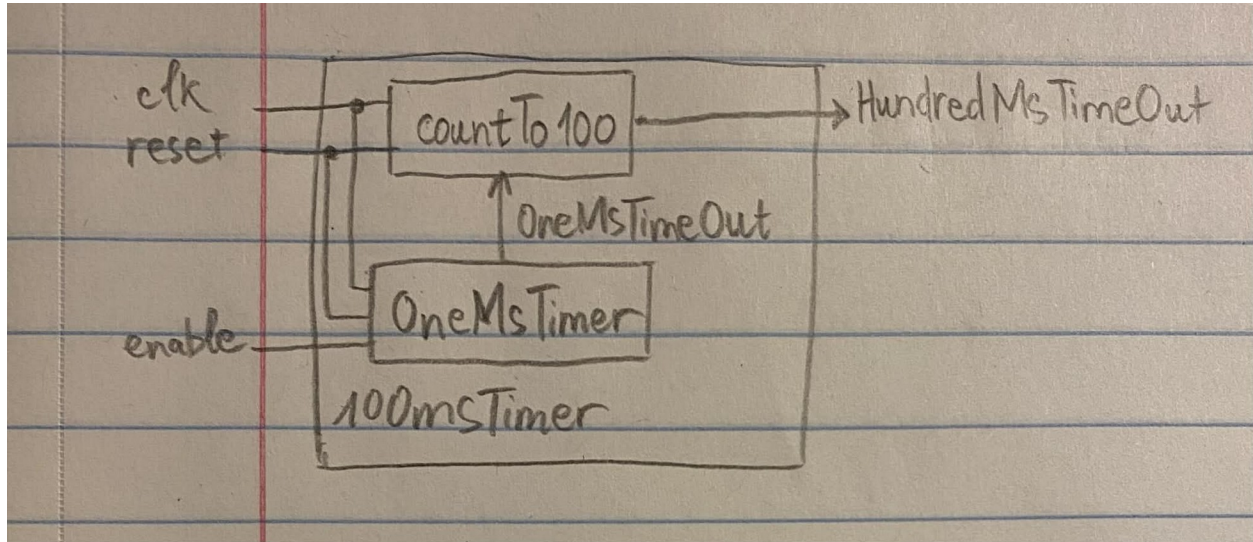
- enable — enables the timing operation
- clk — system clock

- reset — active-low reset (resets all counters)

Output:

- OneSecTimeOut — asserted for one clock cycle every 1 second

2.11. countTo100



countTo100 module counts incoming **1 ms timeout pulses** and generates a **100 ms timeout signal**. When the count reaches its limit, it resets and produces a one-cycle output pulse.

(Note: Counter size is reduced for simulation purposes.)

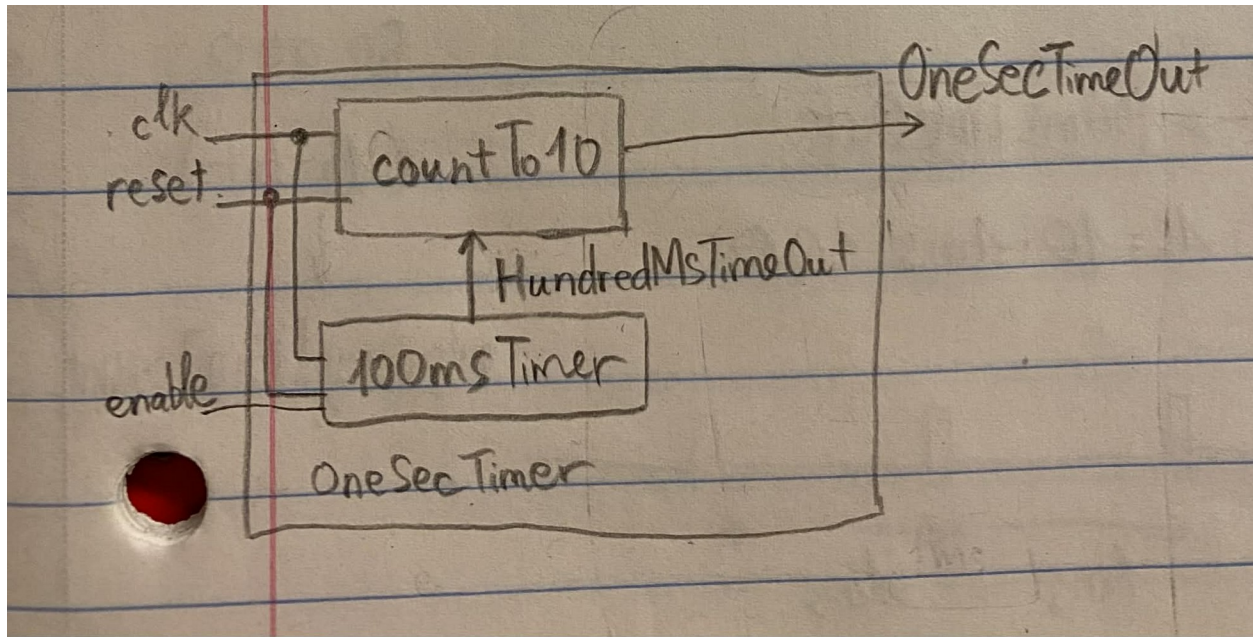
Inputs:

- OneMsTimeOut — input pulse every 1 ms
- clk — system clock
- reset — active-low reset

Outputs:

- HundredMsTimeOut — asserted for one clock cycle every 100 ms
- [1:0] Count_out — current counter value

2.12. countTo10



countTo10 module counts 100 ms timeout pulses and generates a 1 second timeout signal. Once the count limit is reached, it resets and asserts **OneSecTimeOut** for one clock cycle.

(Note: Counter size is reduced for simulation purposes.)

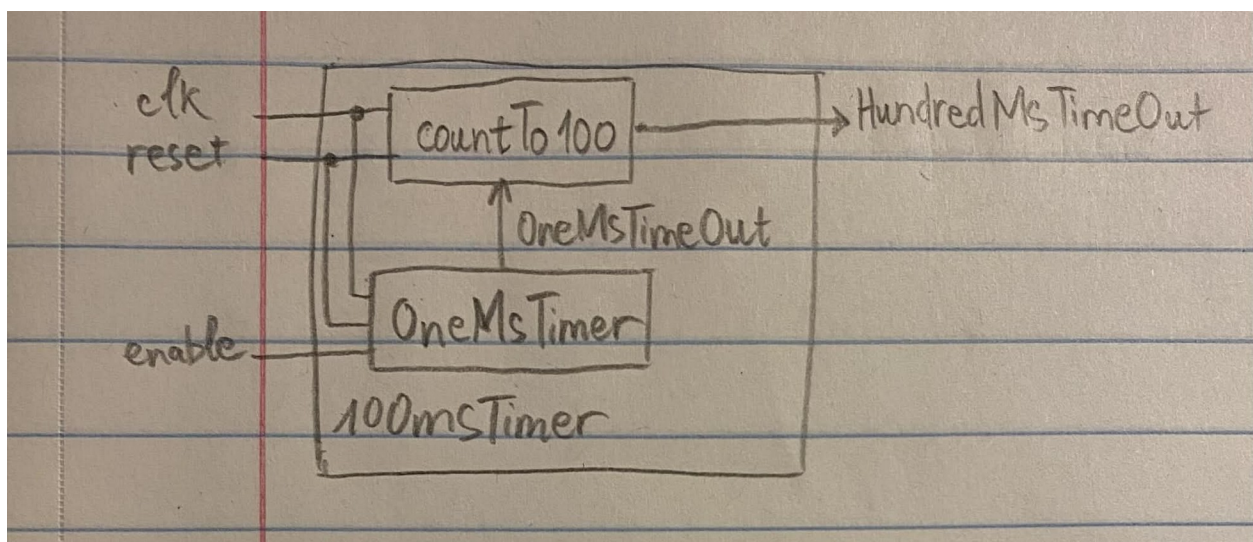
Inputs:

- **HundredMsTimeOut** — input pulse every 100 ms
- **clk** — system clock
- **reset** — active-low reset

Outputs:

- **OneSecTimeOut** — asserted for one clock cycle every 1 second
- **[1:0] Count_out** — current counter value

2.13 OneMsTimer



OneMsTimer module generates a 1 millisecond timeout pulse by counting clock cycles when enabled. When the internal counter reaches its maximum value, it resets and asserts OneMsTimeOut for one clock cycle.

(Note: Counter size is reduced for simulation purposes.)

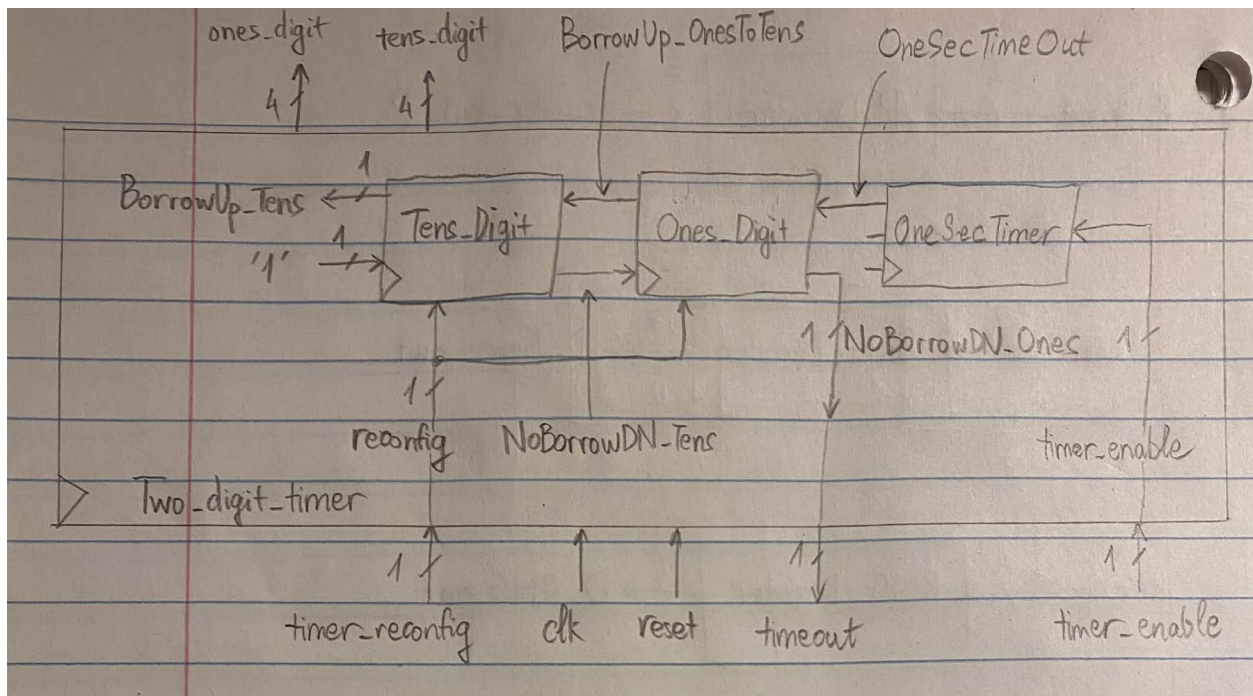
Inputs:

- enable — enables counting
- clk — system clock
- reset — active-low reset

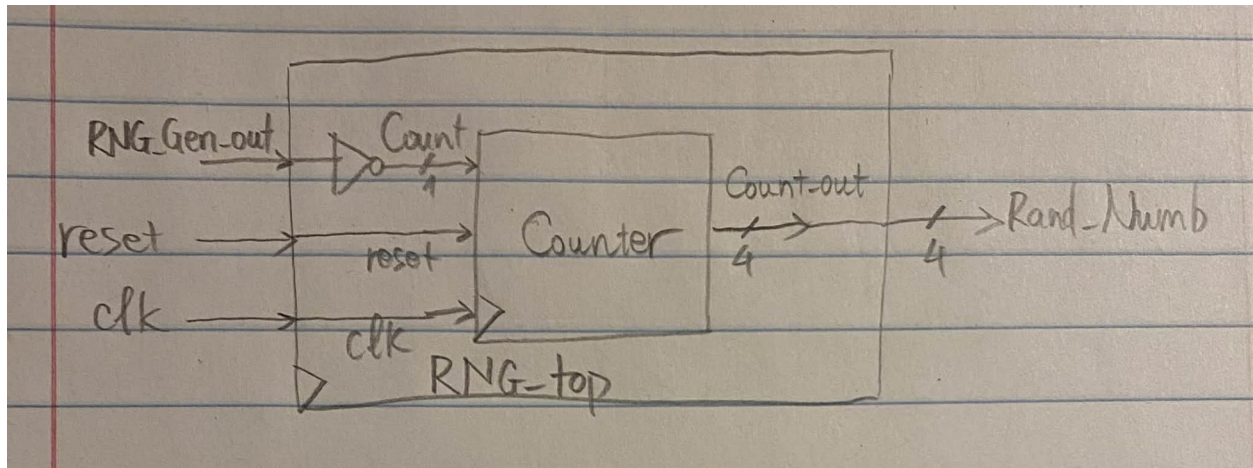
Outputs:

- OneMsTimeOut — goes high for one clock cycle when count completes
- [2:0] Count_out — current counter value

2.14 Two Digit Timer



2.15 RNG_top



3. Simulation Results

3.1. Seven Segment Decoder



Figure 3: 7-segment decoder simulation results

The 7-segment decoder simulation shows all sixteen cases of a 4-bit binary input number from the FPGA's slide switches and corresponding 7-bit output numbers that display indexed segments on 7-segment displays. This test input number is declared `decoder_in_s` and test output number `decoder_out_s`.

Sixteen input test cases are from 0 to F (hex) and corresponding output cases are shown as follows (numbers are in binary):

```

decoder_in_s = 0000; decoder_out_s = 1000000
decoder_in_s = 0001; decoder_out_s = 1111001
decoder_in_s = 0010; decoder_out_s = 0100100
decoder_in_s = 0011; decoder_out_s = 0110000
decoder_in_s = 0100; decoder_out_s = 0011001
decoder_in_s = 0101; decoder_out_s = 0010010
decoder_in_s = 0110; decoder_out_s = 0000010
decoder_in_s = 0111; decoder_out_s = 1111000
decoder_in_s = 1000; decoder_out_s = 0000000
decoder_in_s = 1001; decoder_out_s = 0010000
decoder_in_s = 1010; decoder_out_s = 0001000
decoder_in_s = 1011; decoder_out_s = 0000011
decoder_in_s = 1100; decoder_out_s = 0100111
decoder_in_s = 1101; decoder_out_s = 0100001
decoder_in_s = 1110; decoder_out_s = 0000110
decoder_in_s = 1111; decoder_out_s = 0001110
default; decoder_out_s = 7'b1000000

```

3.2. Four Bit Adder

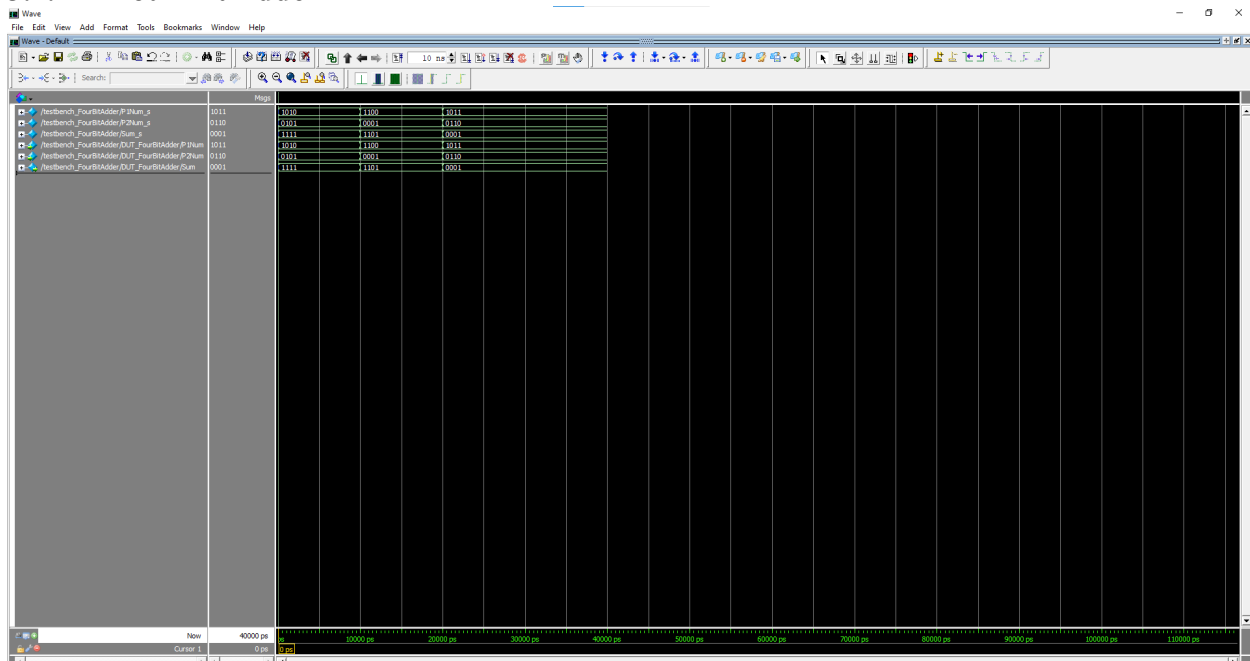
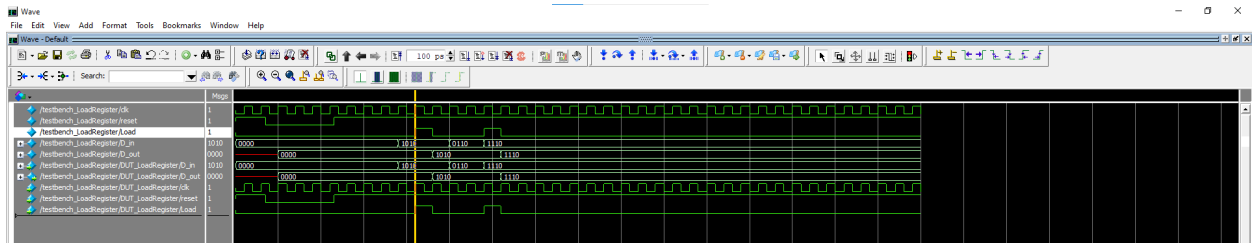


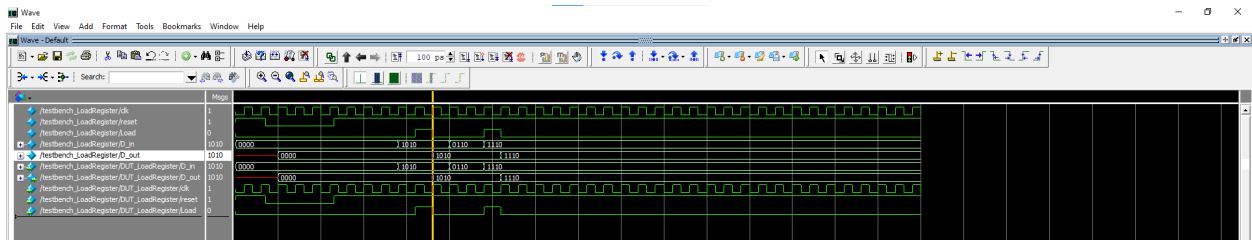
Figure 4: 4-bit adder simulation results

The 4-bit adder simulation shows three sets of test 4-bit input binary numbers from the FPGA's slide switches and corresponding sum of each set. These three sets of test input numbers include a case of matching number, which the sum equals 1111 in binary, and two cases of non-matching

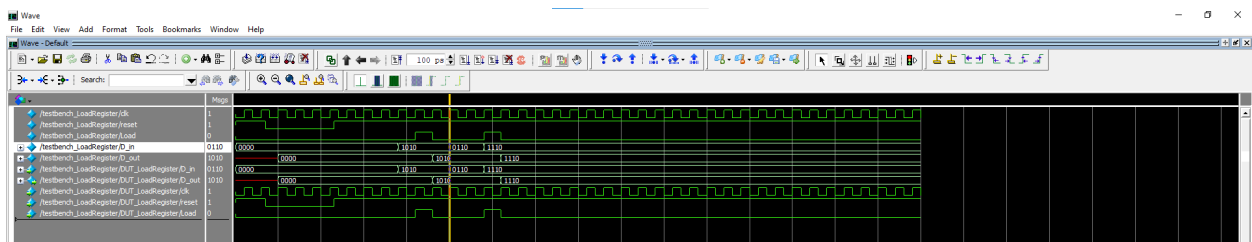
D_in is then loaded with the value of 4'b1010.



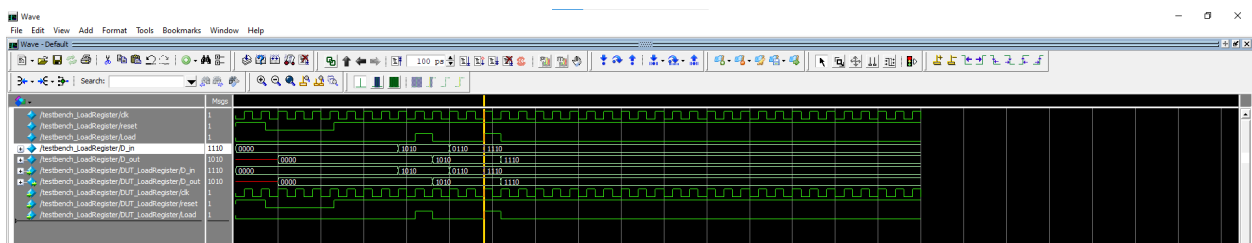
Load signal is high (loading enabled).



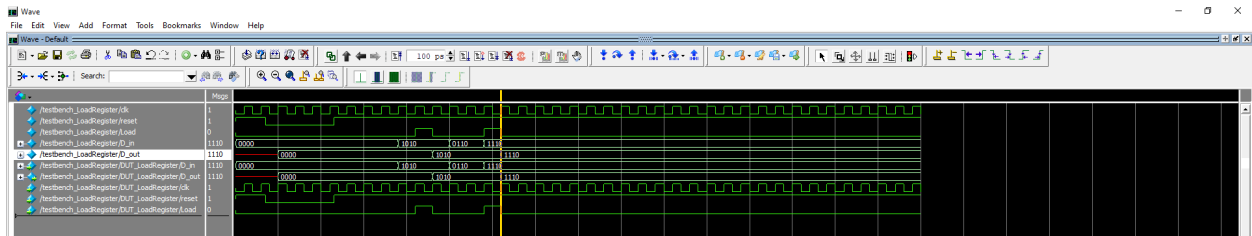
D_out is loaded with D_in's value of 4'b1010 at the following rising edge of clk signal.



D_in is loaded with the value of 4'b0110. D_out isn't loaded with this value of D_in because Load signal isn't raised.

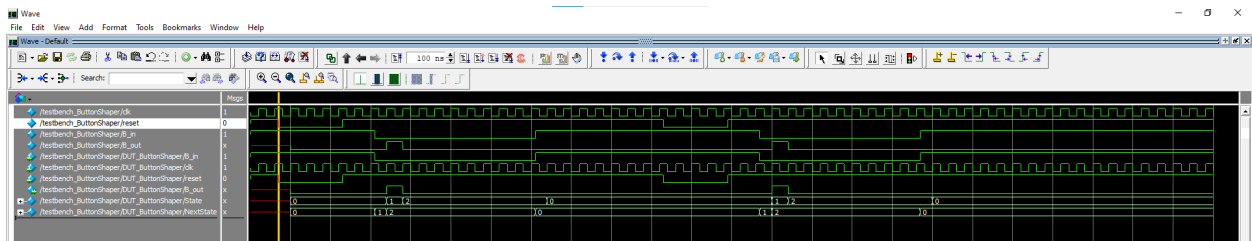


D_in is loaded with the value of 4'b1110 and the Load signal is raised at the same time.

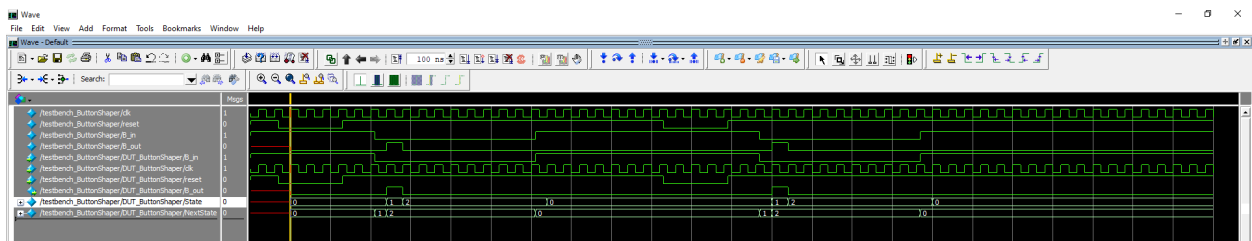


D_out is loaded with D_in's value of 4'b1110 at the following rising edge of clk signal.

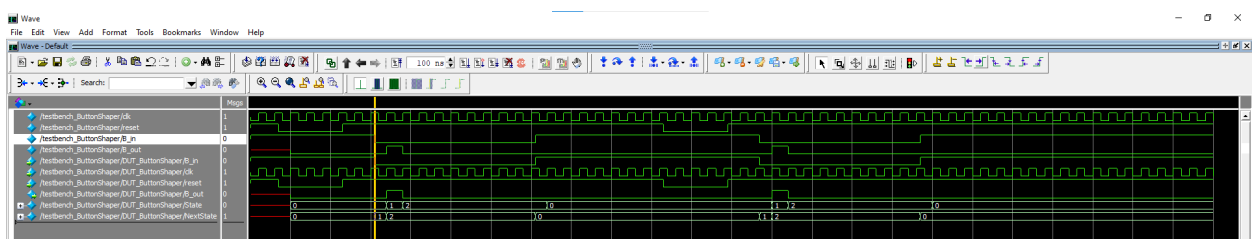
3.4. Button Shaper



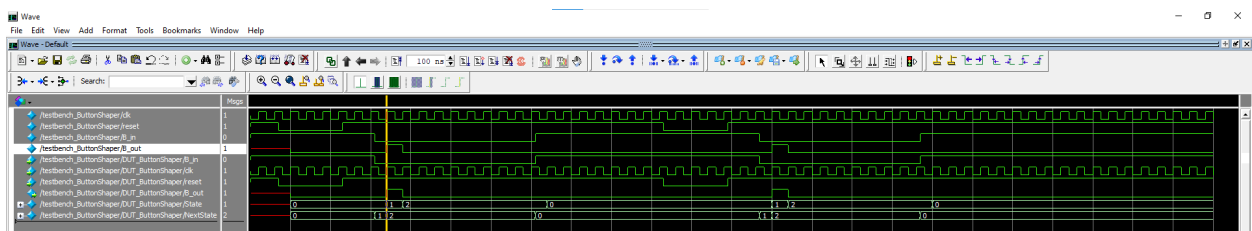
reset signal is pulled low/ active low (reset button is pushed).



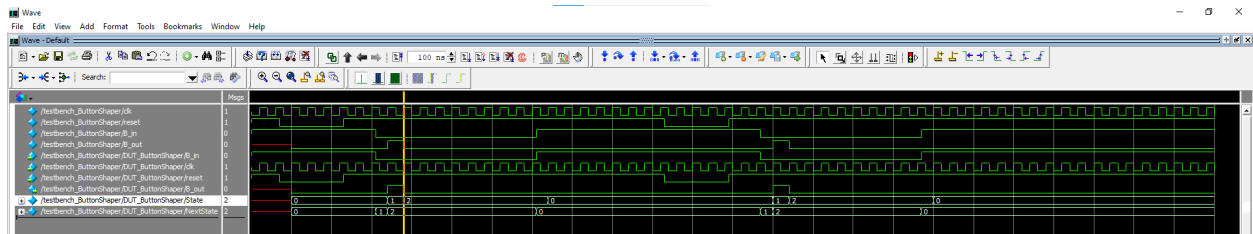
It gets into State 1 at the following rising edge of clk signal after reset signal is pulled low.



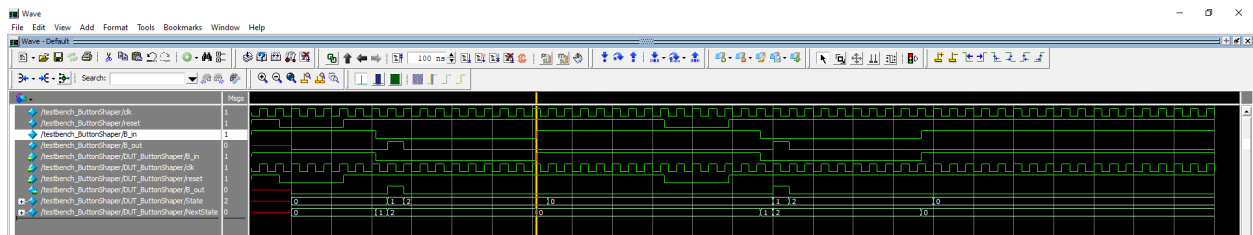
B_in is pulled low/ active low (a player's input button is pushed).



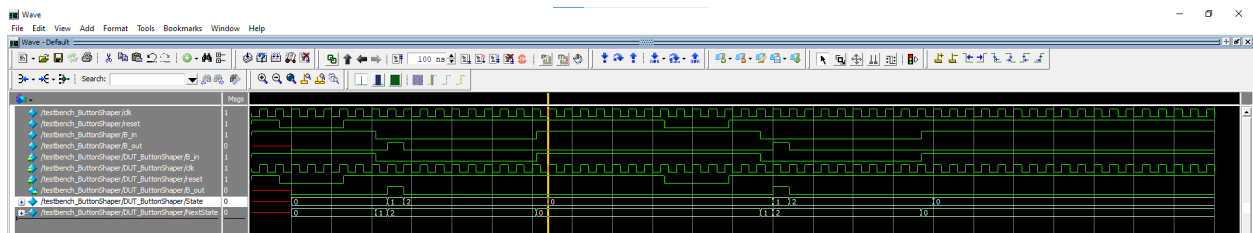
B_out is generated as an active high one-clock-cycle pulse signal at the rising edge of clk signal and it gets into State 2.



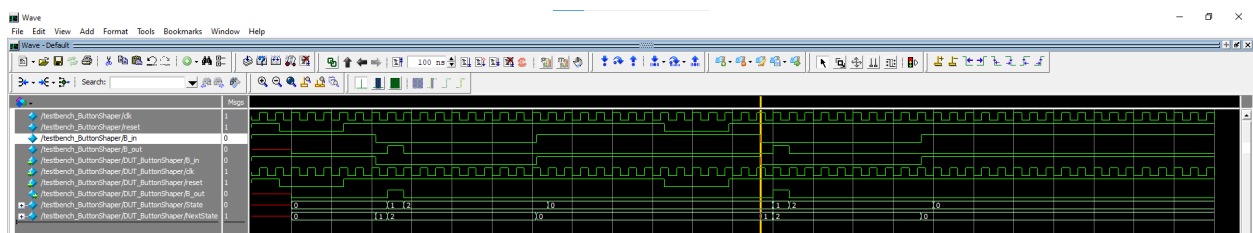
B_out is raised for exactly one clock cycle. At this point, it gets into State 2 waiting for the physical player's input button to be released.



B_in is raised (player's button is released).



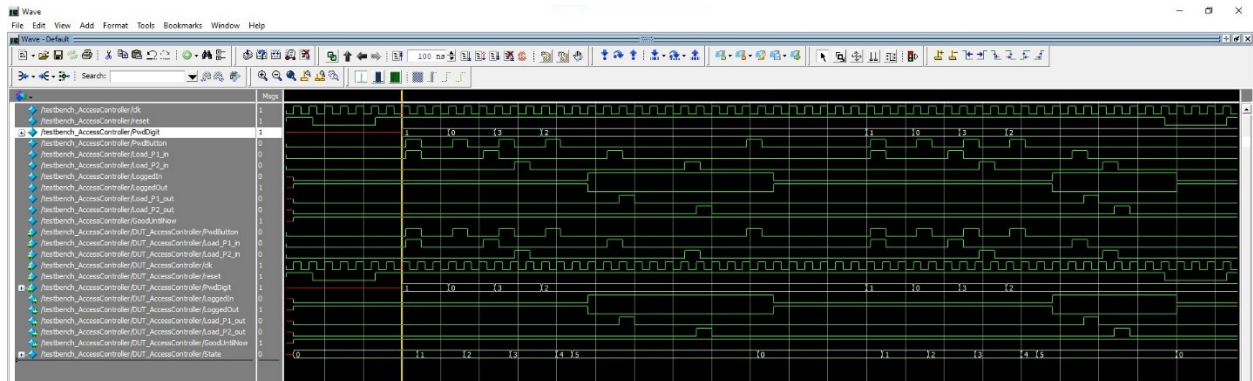
It gets back to State 0 at the following rising edge of clk signal as the player's button is released.



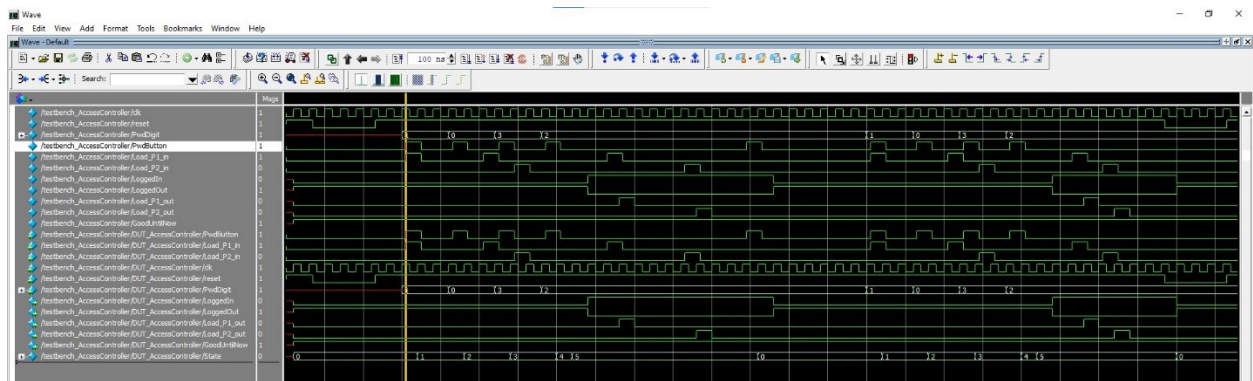
The remaining waveforms repeat the cycle again with similar procedures.

3.5. Access Controller

3.5.1. Correct password condition simulation



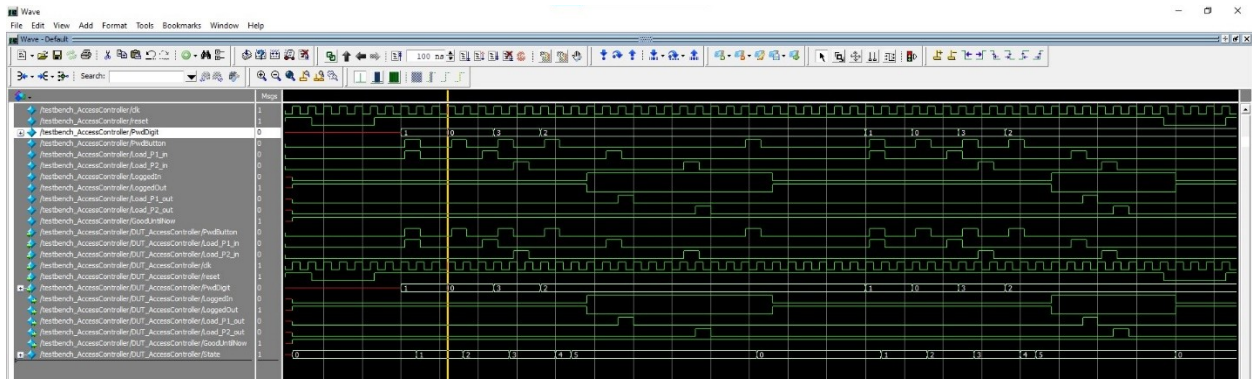
GoodUntilNow signal is raised initially. First digit of the password is loaded into PwdDigit (1'd1 or 4'b0001).



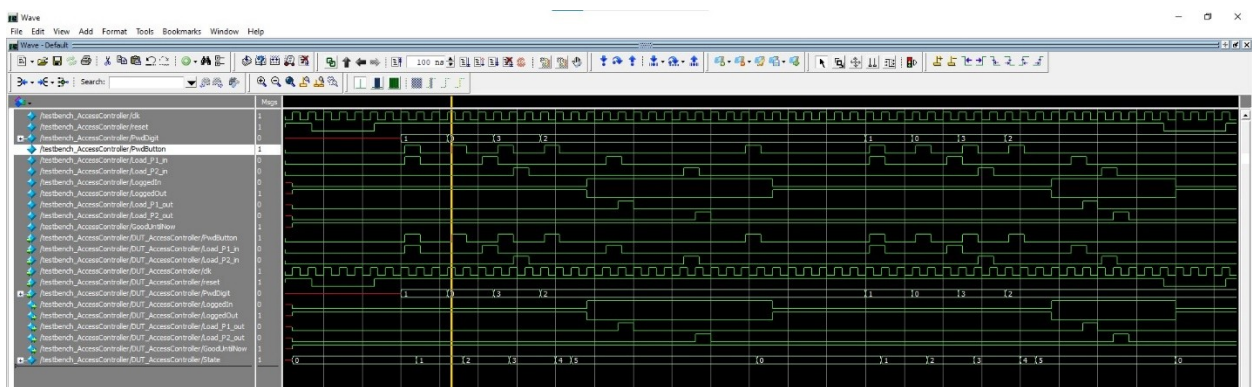
The first one-clock-cycle pulse signal PwdButton enables the loading of the first password digit. At the same time, Load_P1_in is raised indicating Player 1 pushes their LOAD button but Load_P1_out stays low. This means that players can't load their input numbers before entering the correct password.



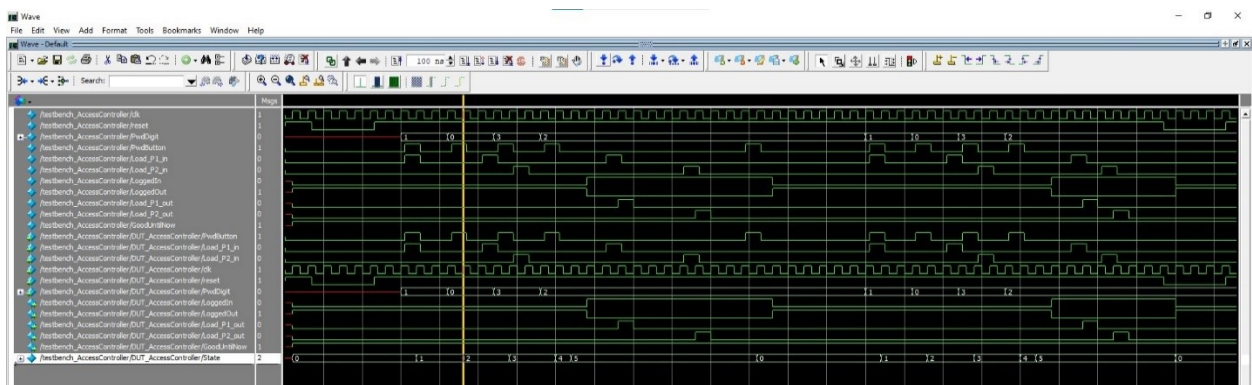
State changes from 0 to 1 at the next rising edge of clk signal. GoodUntilNow stays high meaning the first password digit is correct.



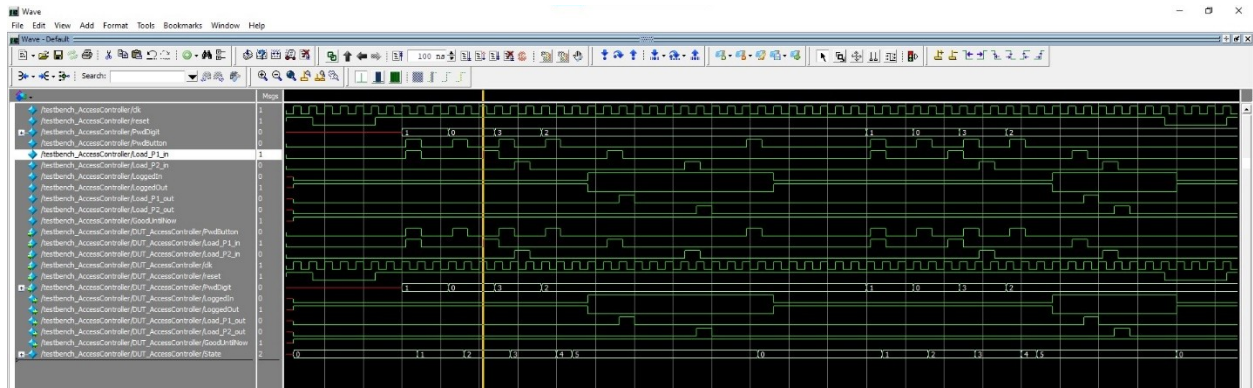
Second digit of the password is loaded into PwdDigit (1'd0 or 4'b0000).



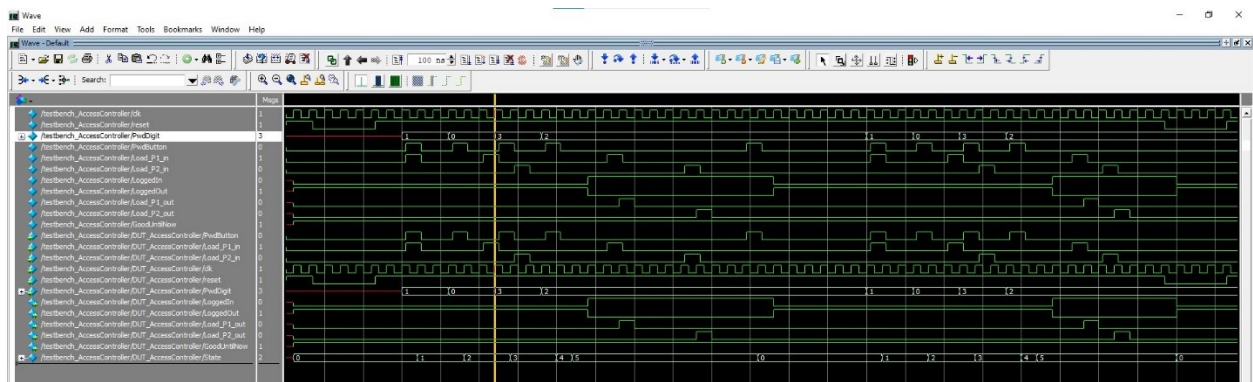
The second one-clock-cycle pulse signal PwdButton enables the loading of the second password digit.



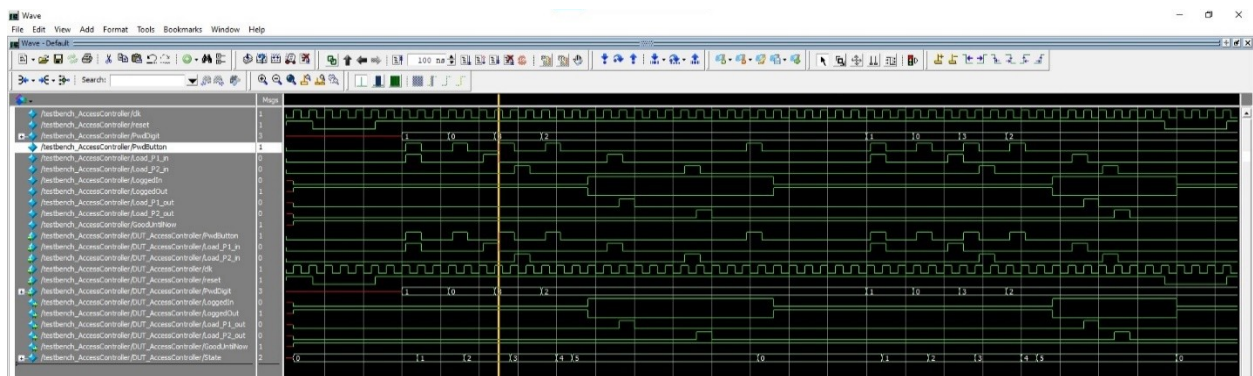
State changes from 1 to 2 at the next rising edge of clk signal. GoodUntilNow still stays high meaning the second password digit is correct.



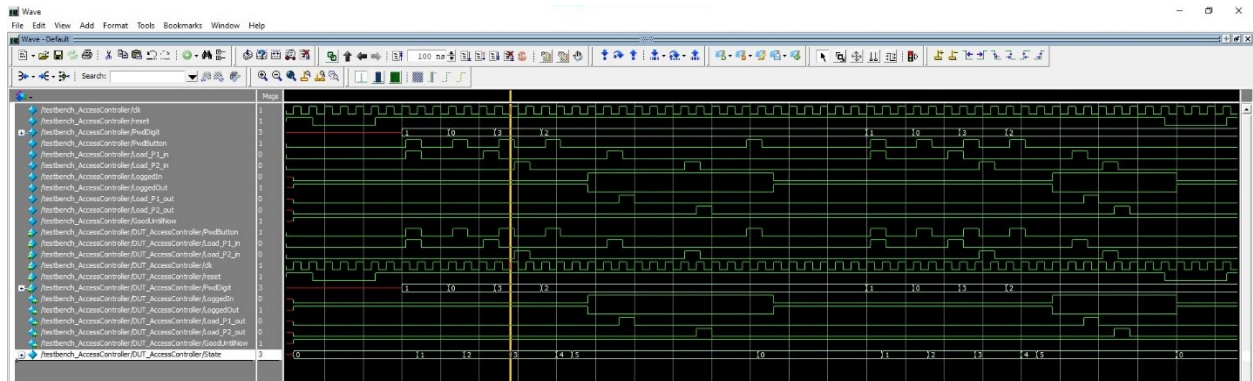
Load_P1_in is raised again meaning Player 1 pushes their LOAD button but loading permission Load_P1_out is still low/denied because the password hasn't been entered fully and/or correctly.



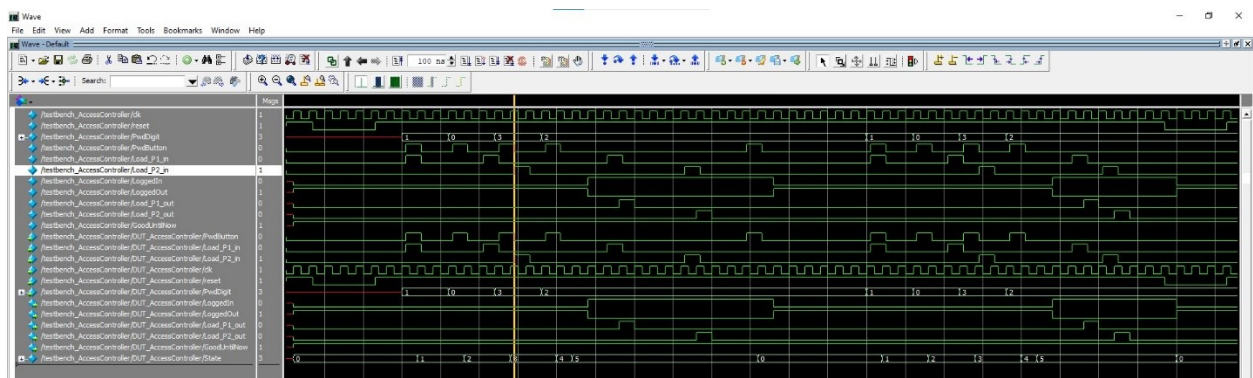
Third digit of the password is loaded into PwdDigit (1'd3 or 4'b0011).



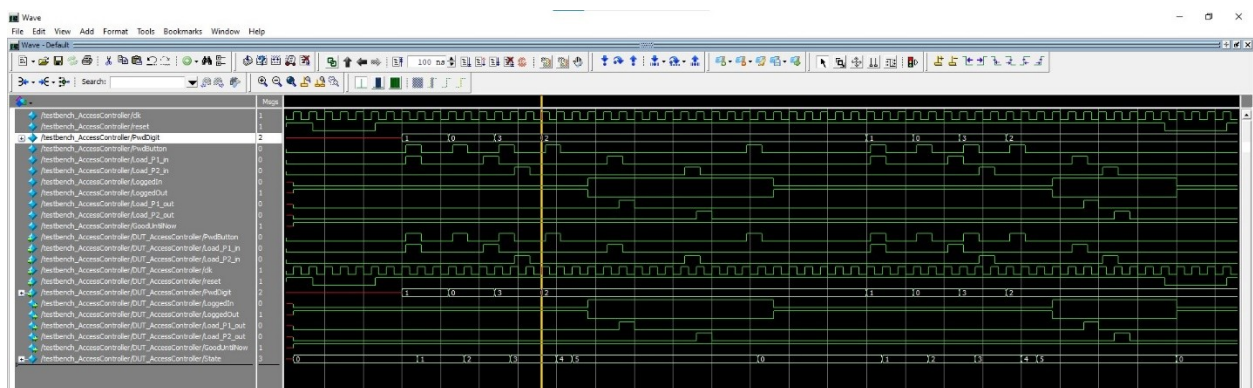
The third one-clock-cycle pulse signal PwdButton enables the loading of the third password digit.



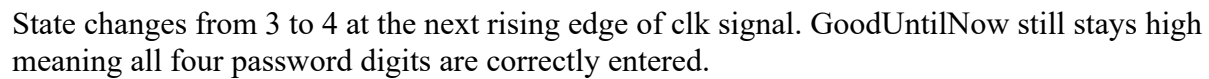
State changes from 2 to 3 at the next rising edge of clk signal. GoodUntilNow still stays high meaning the third password digit is correct.

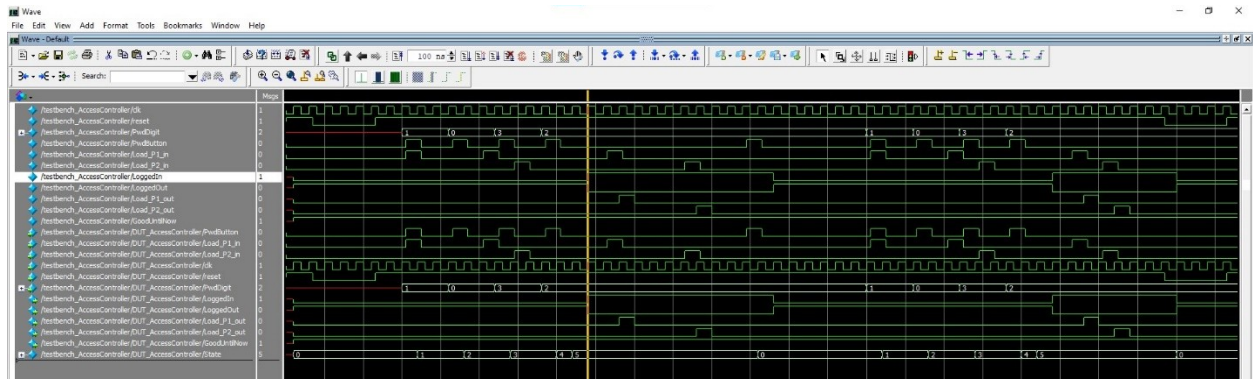


Load_P2_in is raised meaning Player 2 pushes their LOAD button but loading permission Load_P2_out is still low/denied because the password hasn't been entered fully and/or correctly.

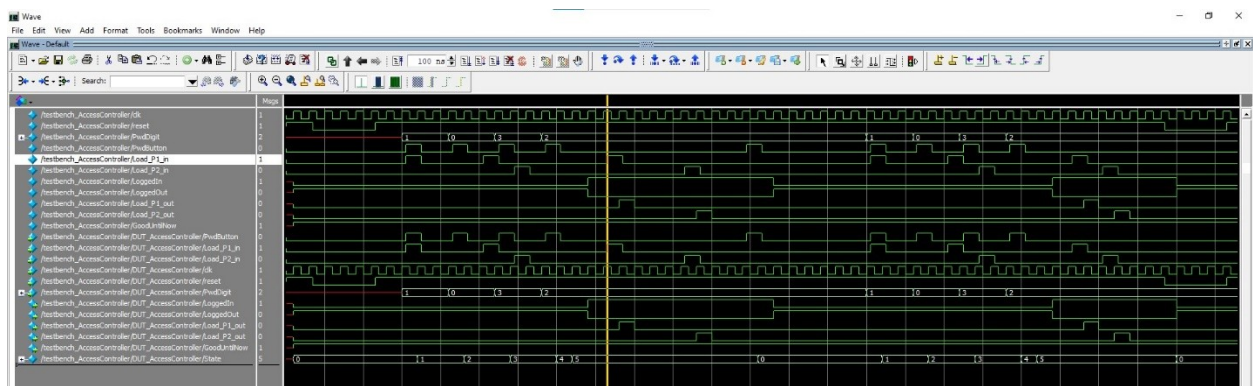


Last digit of the password is loaded into PwdDigit (1'd2 or 4'b0010).

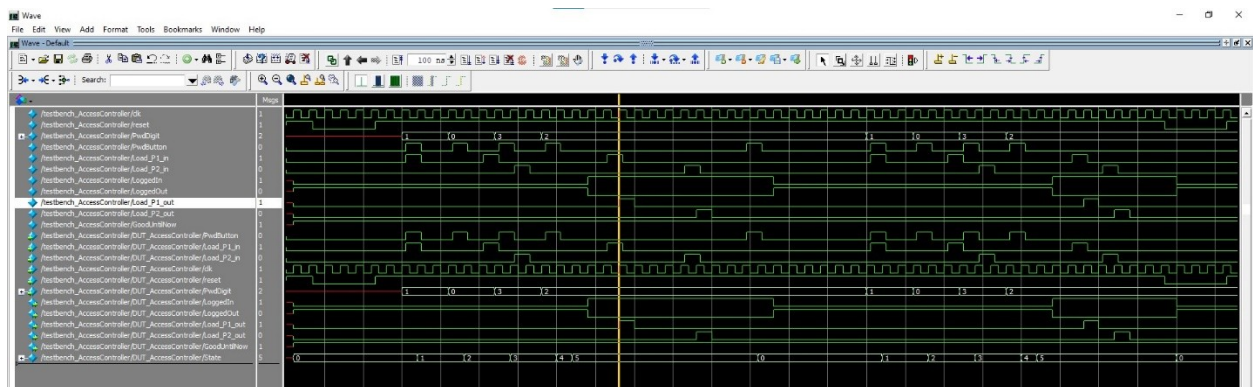




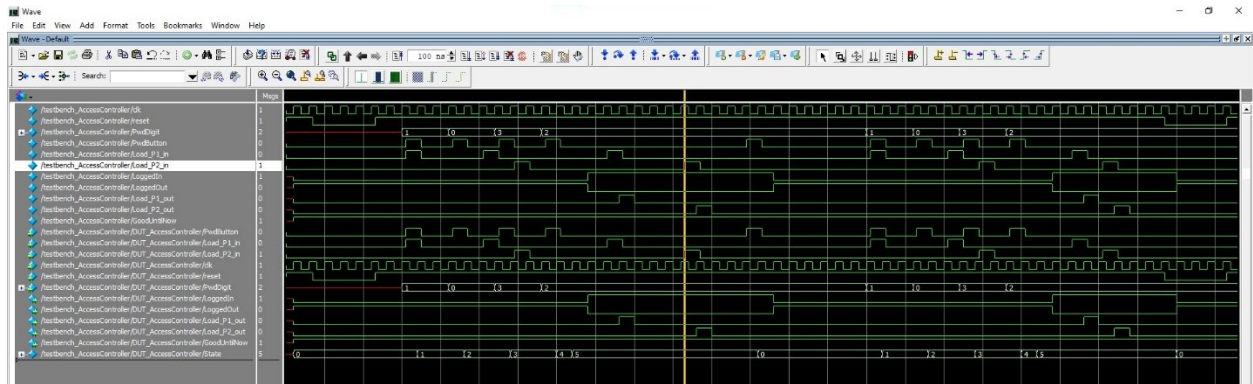
LoggedIn is raised at the next rising edge of clk signal and LoggedOut is pulled low. This indicates that the password is correct and players are logged in.



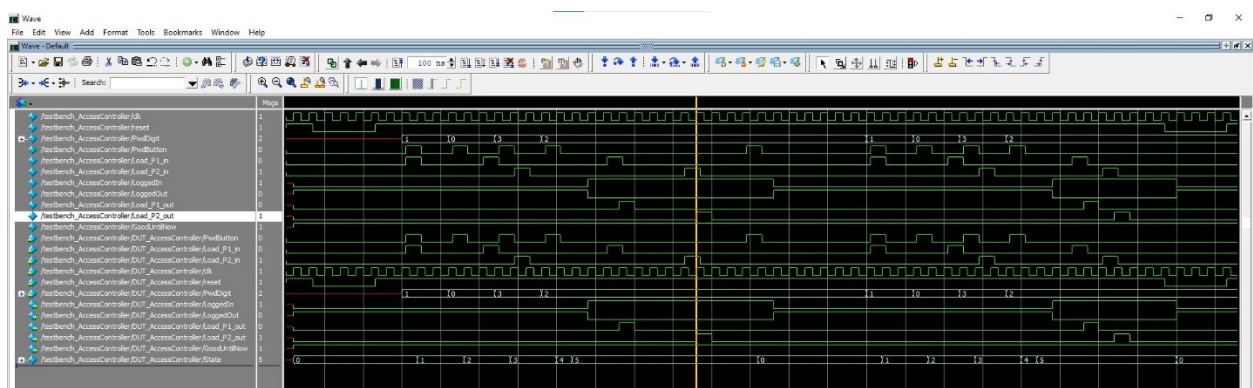
Player 1 now pushes their LOAD button again as Load_P1_in is raised.



Load_P1_out is raised at the next rising edge of clk signal. This means the players can load their numbers now as they're logged in.

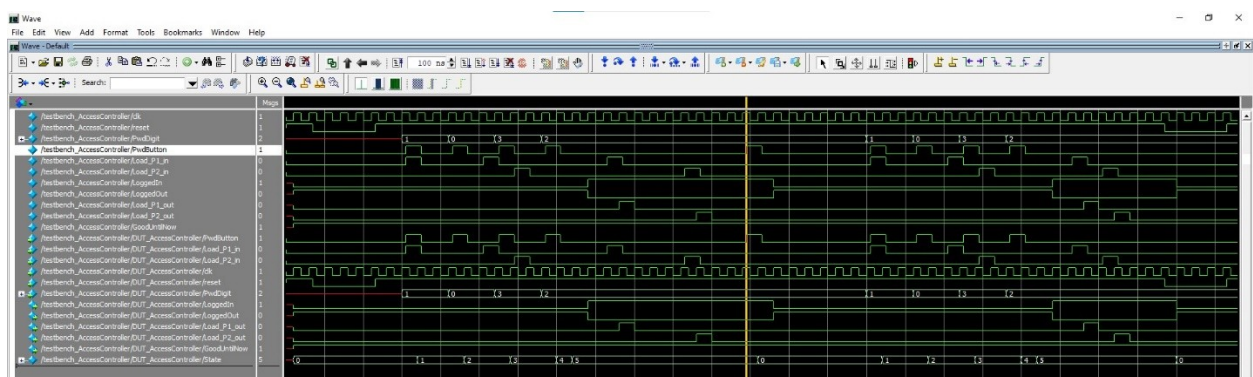


Player 2 pushes their LOAD button as Load_P2_in is raised.

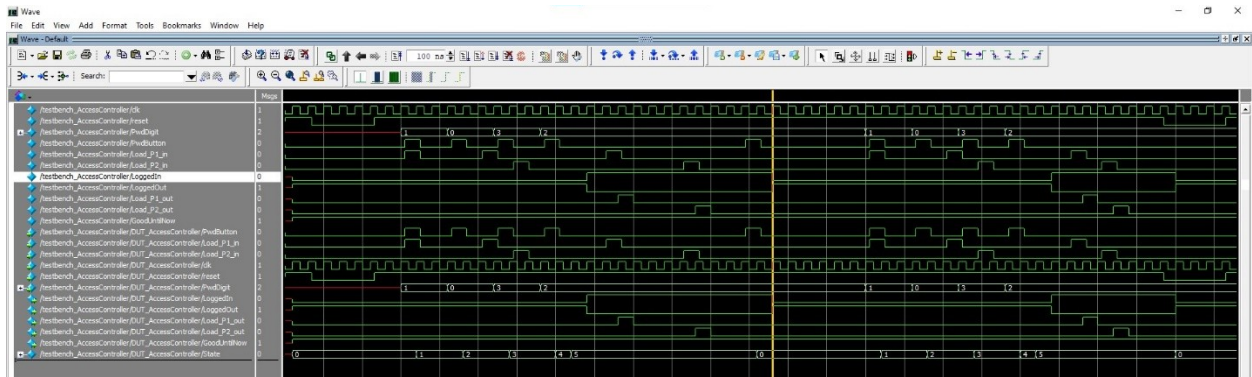


Load_P2_out is raised at the next rising edge of clk signal.

Bonus feature:



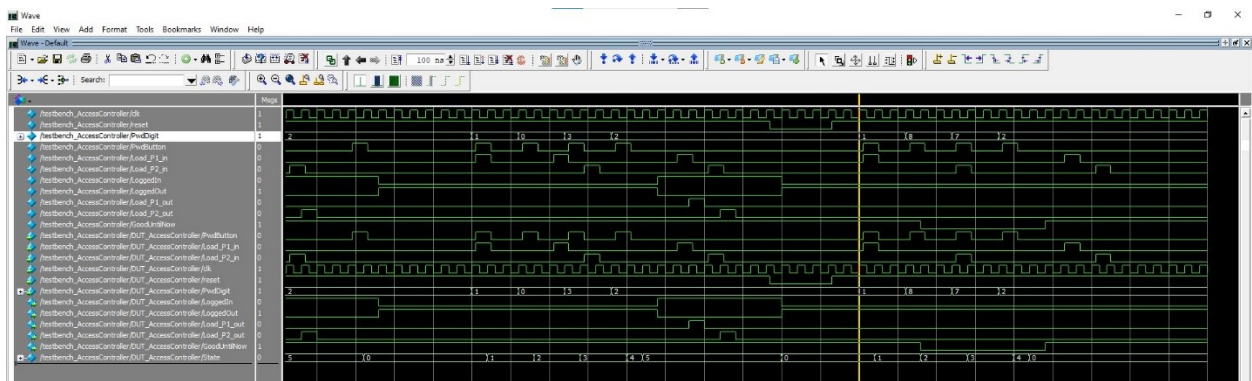
The password load button is pushed when players are logged in as PwdButton is raised.



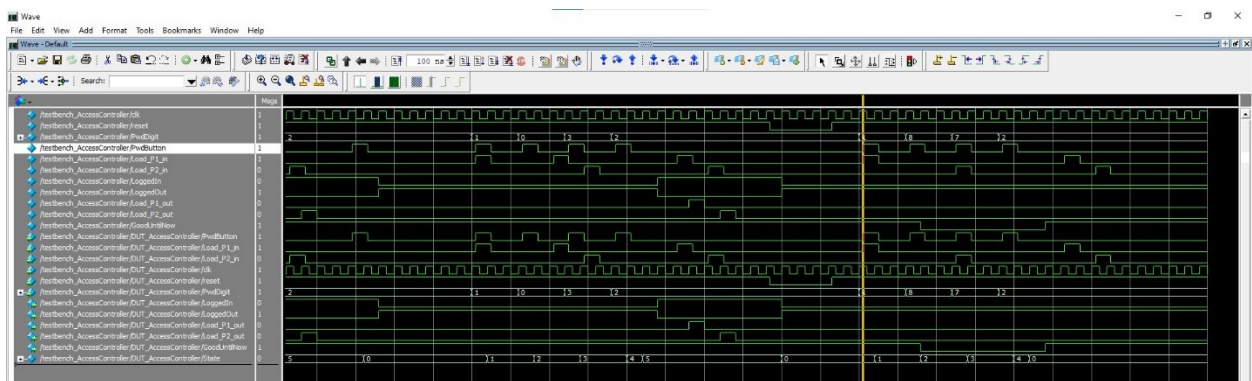
LoggedIn is pulled low and LoggedOut is raised at the next rising edge of clk signal. This means players are logged out again after the previous pass load button push.

The next waveforms indicate that players try to log in again with the same password (1032).

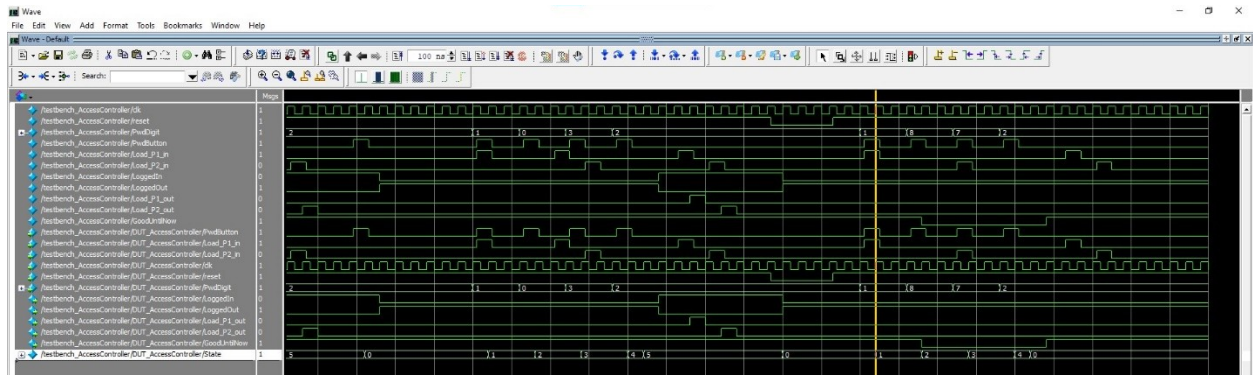
3.5.1. Wrong password condition simulation



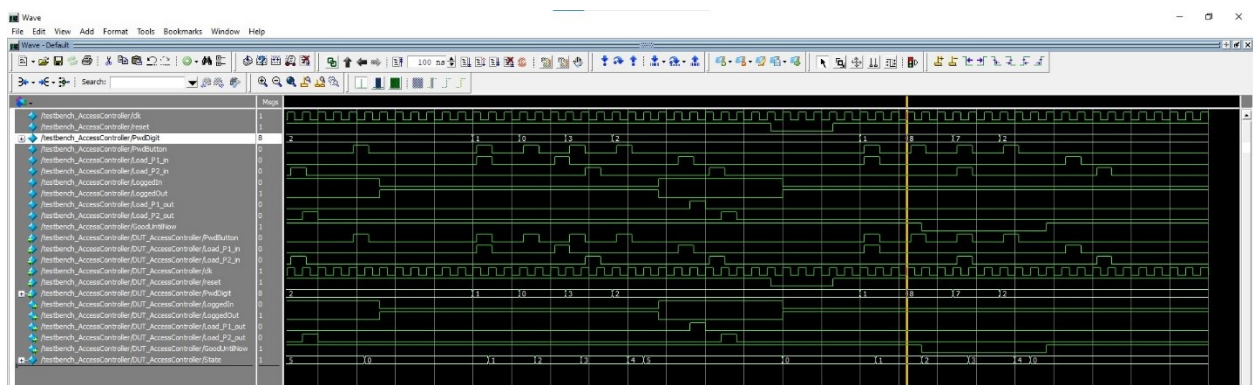
First digit of the password is loaded into PwdDigit (1'd1 or 4'b0001).



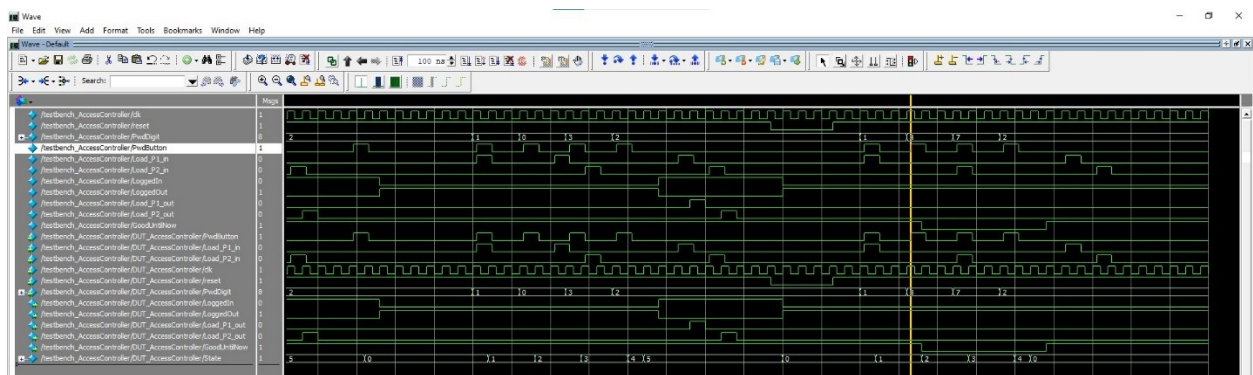
The first one-clock-cycle pulse signal PwdButton enables the loading of the first password digit. At the same time, Load_P1_in is raised indicating Player 1 pushes their LOAD button but Load_P1_out stays low. This means that players can't load their input numbers before entering the correct password.



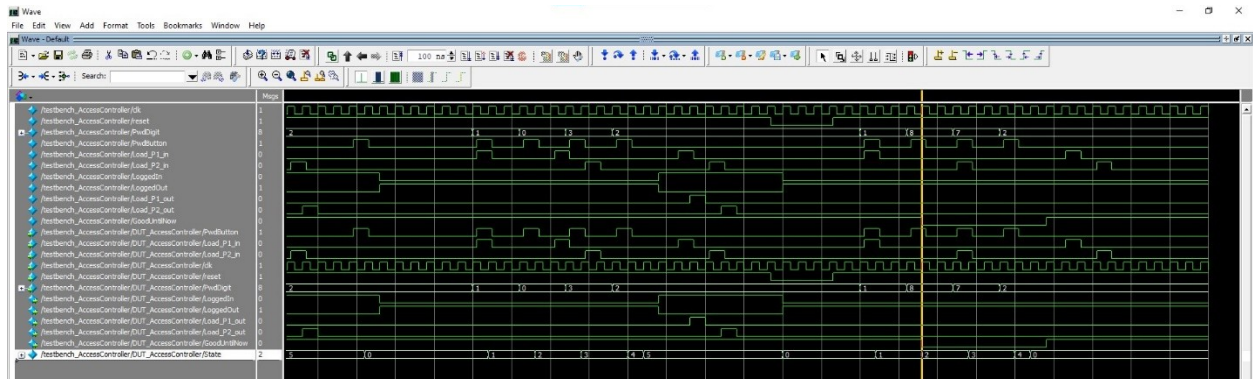
State changes from 0 to 1 at the next rising edge of clk signal. GoodUntilNow stays high meaning the first password digit is correct.



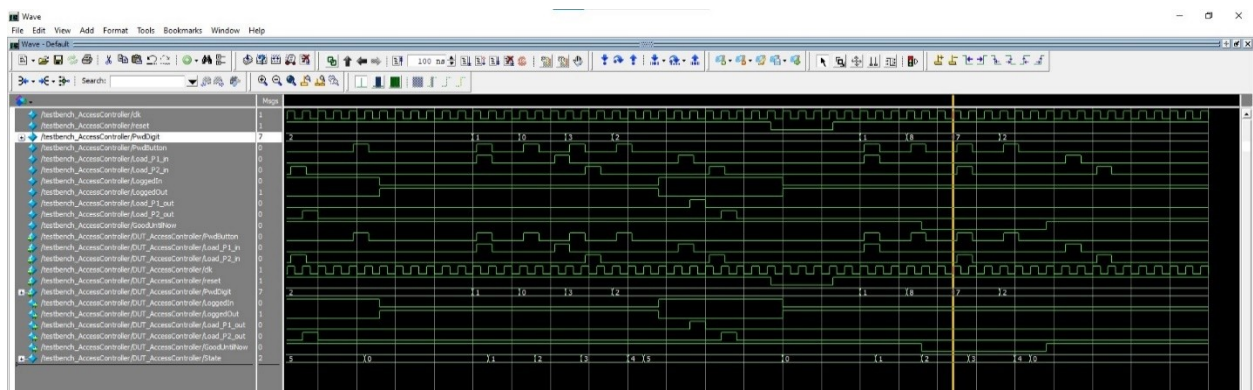
Second digit of the password is loaded into Pwddigit (1'd8 or 4'b1000).



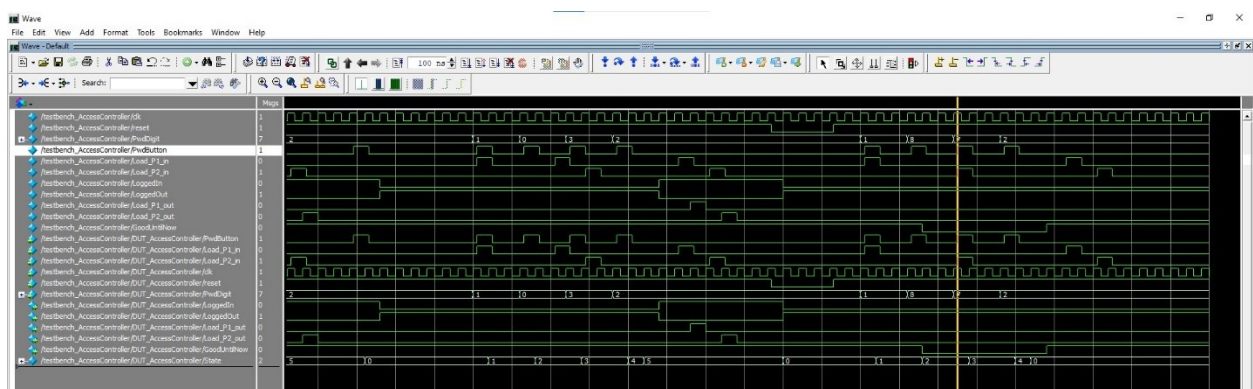
The second one-clock-cycle pulse signal Pwddigit enables the loading of the second password digit.



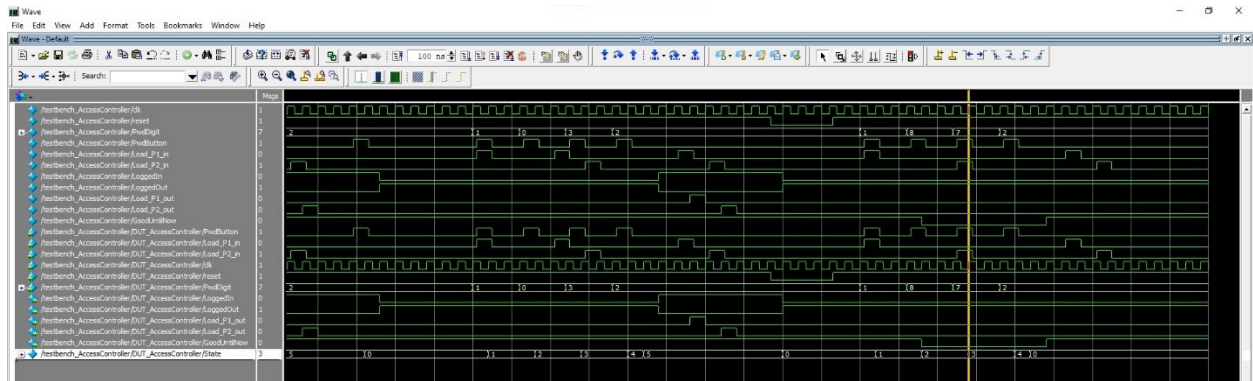
State changes from 1 to 2 at the next rising edge of clk signal. GoodUntilNow is pulled low meaning the second password digit is incorrect.



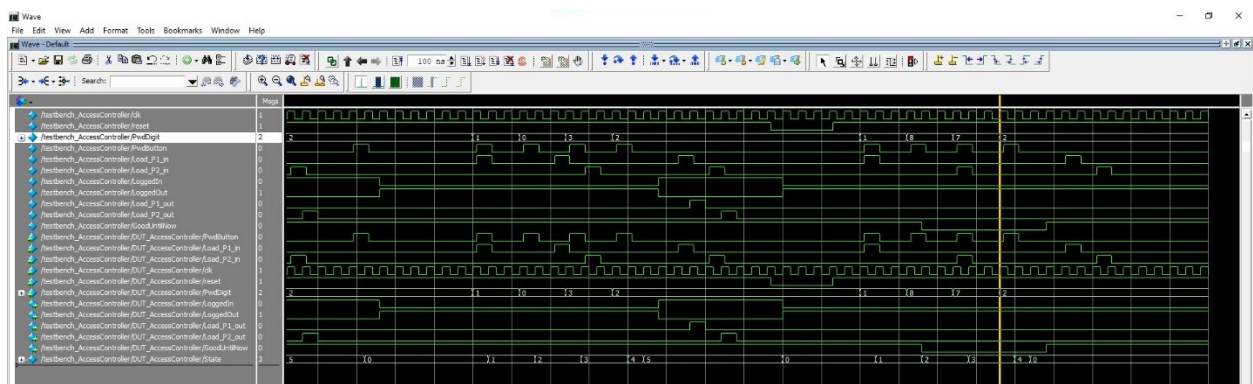
Third digit of the password is loaded into PwdDigit (1'd7 or 4'b0111).



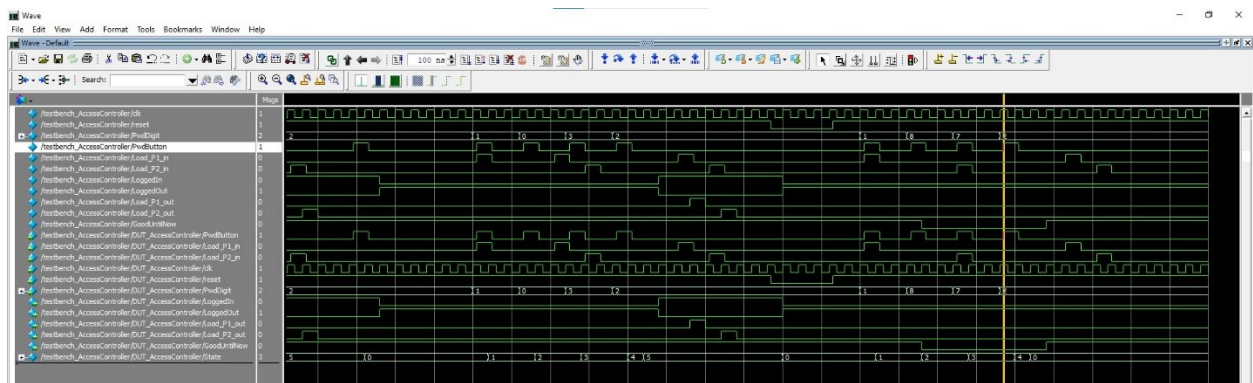
The third one-clock-cycle pulse signal PwdButton enables the loading of the third password digit. At the same time, Load_P2_in is raised indicating Player 2 pushes their LOAD button but Load_P2_out stays low. This means that players can't load their input numbers before entering the correct password.



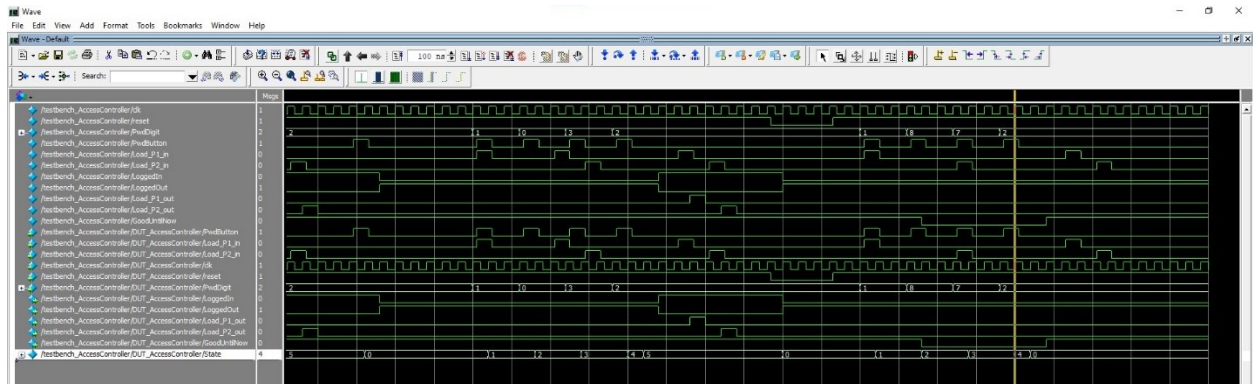
State changes from 2 to 3 at the next rising edge of `clk` signal. `GoodUntilNow` stays low once a wrong digit is entered so players must enter all 4 correct password digits to be logged in.



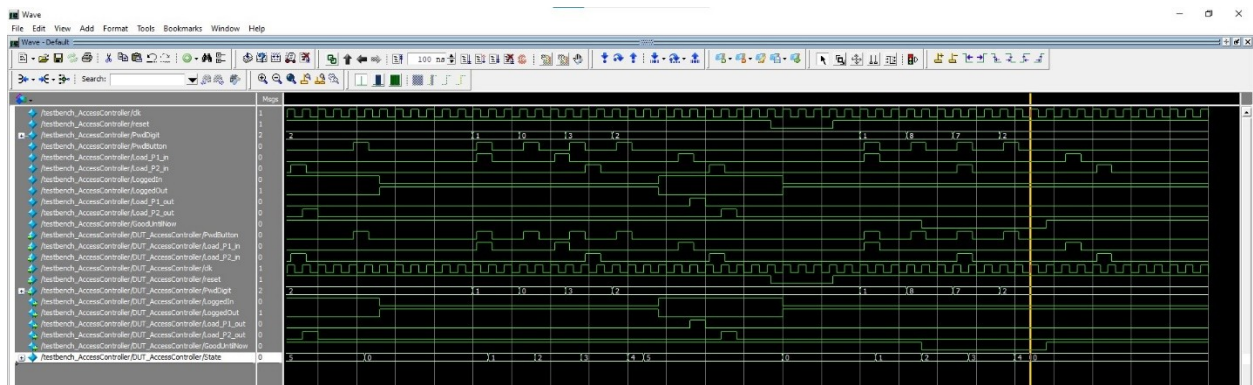
Last digit of the password is loaded into `PwdDigit` (1'd2 or 4'b0010).



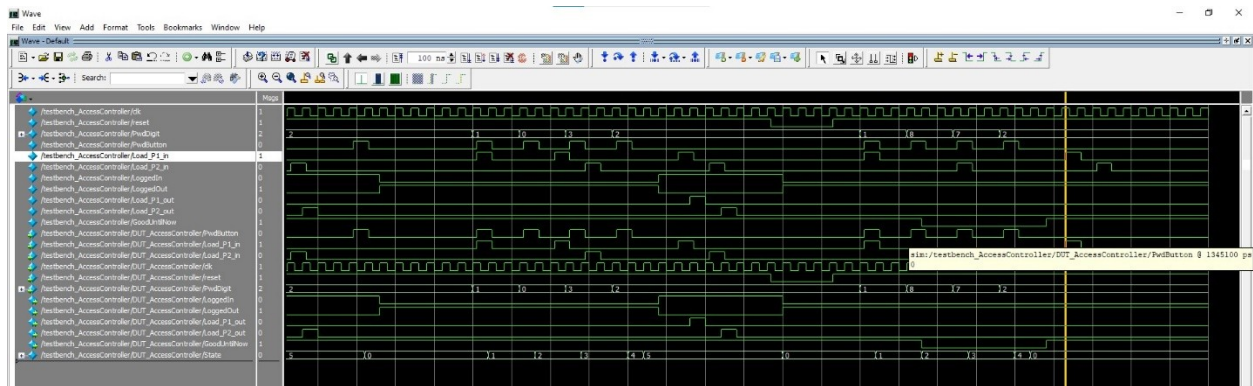
The last one-clock-cycle pulse signal `PwdButton` enables the loading of the last password digit.



State changes from 3 to 4 at the next rising edge of clk signal. GoodUntilNow still stays low even the last password digit is correctly entered.



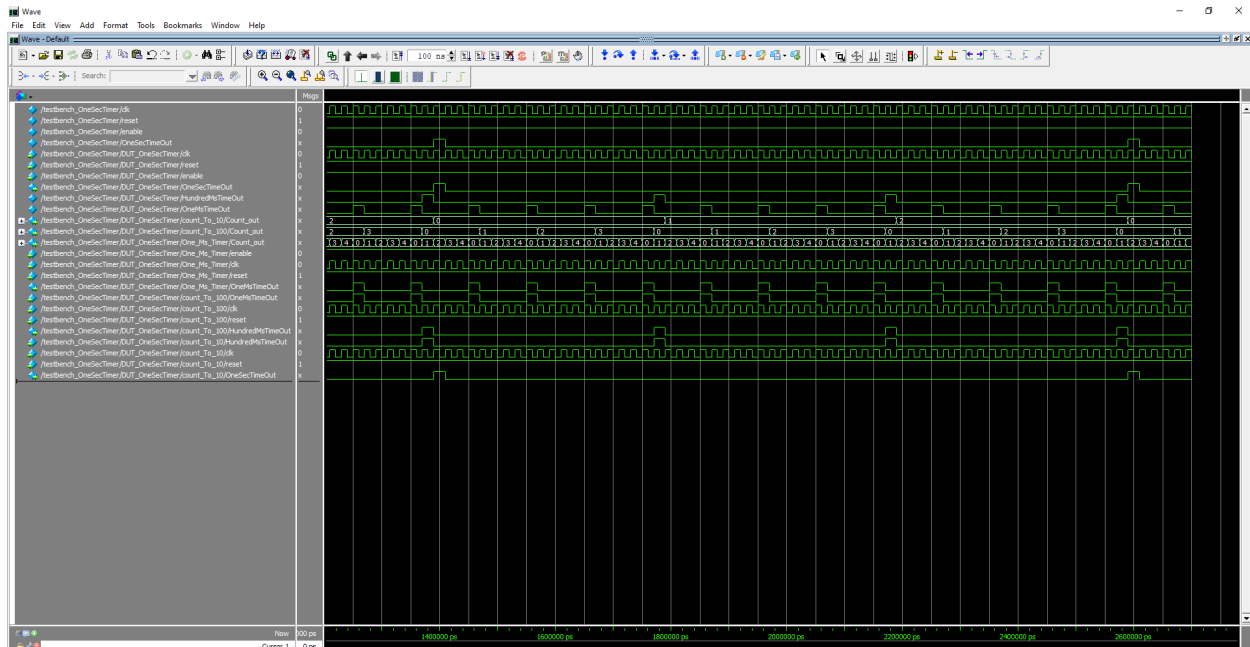
State changes from 4 to 0 at the next rising edge of clk signal. This means the password is incorrectly entered. Players must enter the password again. Through out this process, LoggedOut stays high while LoggedIn stays low the entire time.



Player 1 pushes their LOAD button as Load_P1_in is raised. As can be seen at the next rising edge of clk signal, Load_P1_out stays low because players are not logged in to load their numbers.

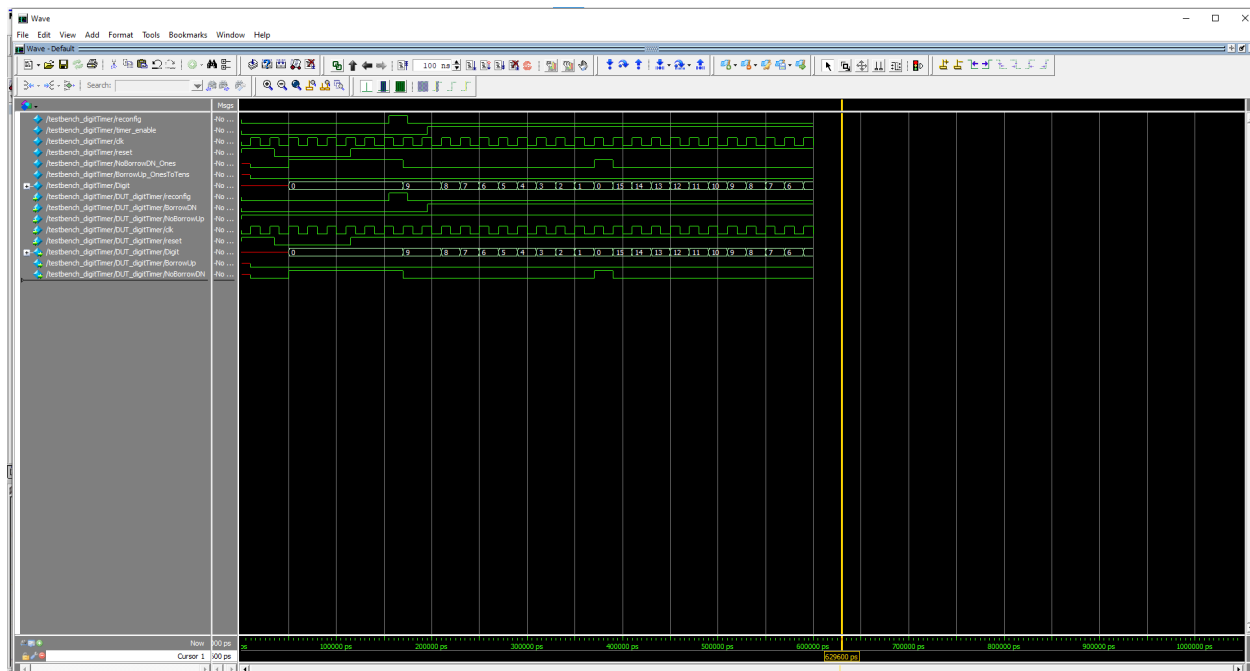


Simulation waveform showing the first pulse of OneSecTimeOut signal

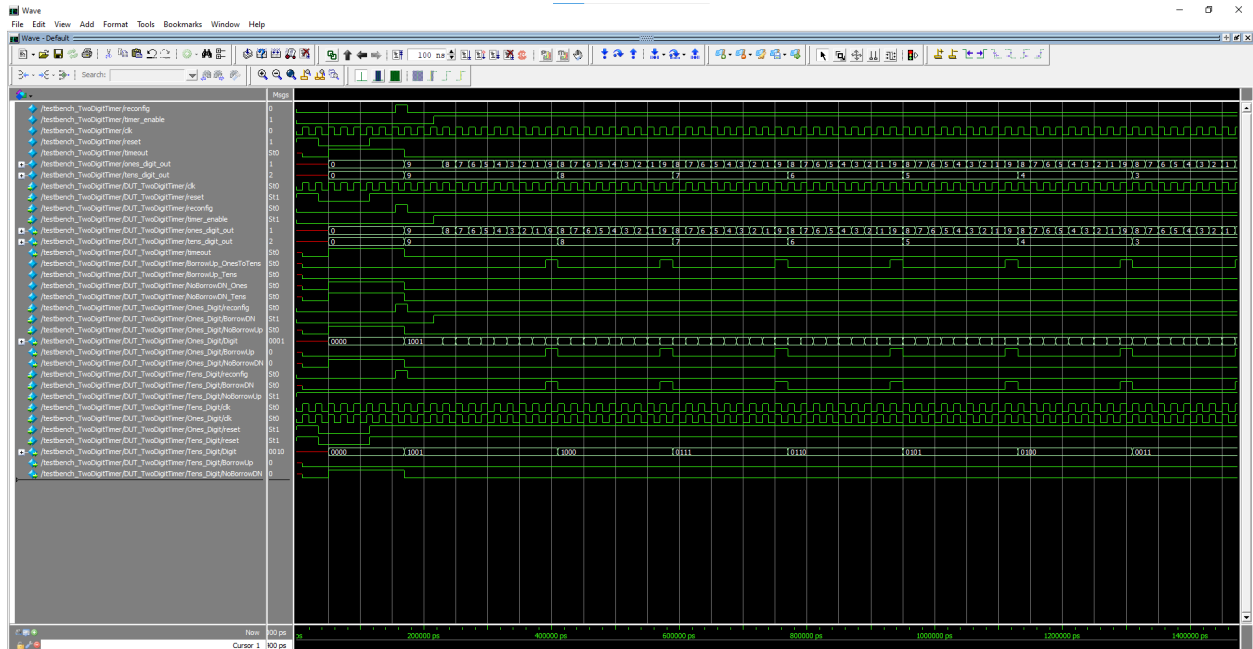


Simulation results show 2 one-clock-cycle pulses of the OneSecTimeout signal. These pulses occur once for every 3 one-clock-cycle pulses of the HundredMsTimeout and every 12 one-clock-cycle pulses of the OneMsTimeout (*Note that this is for simulation purposes only*).

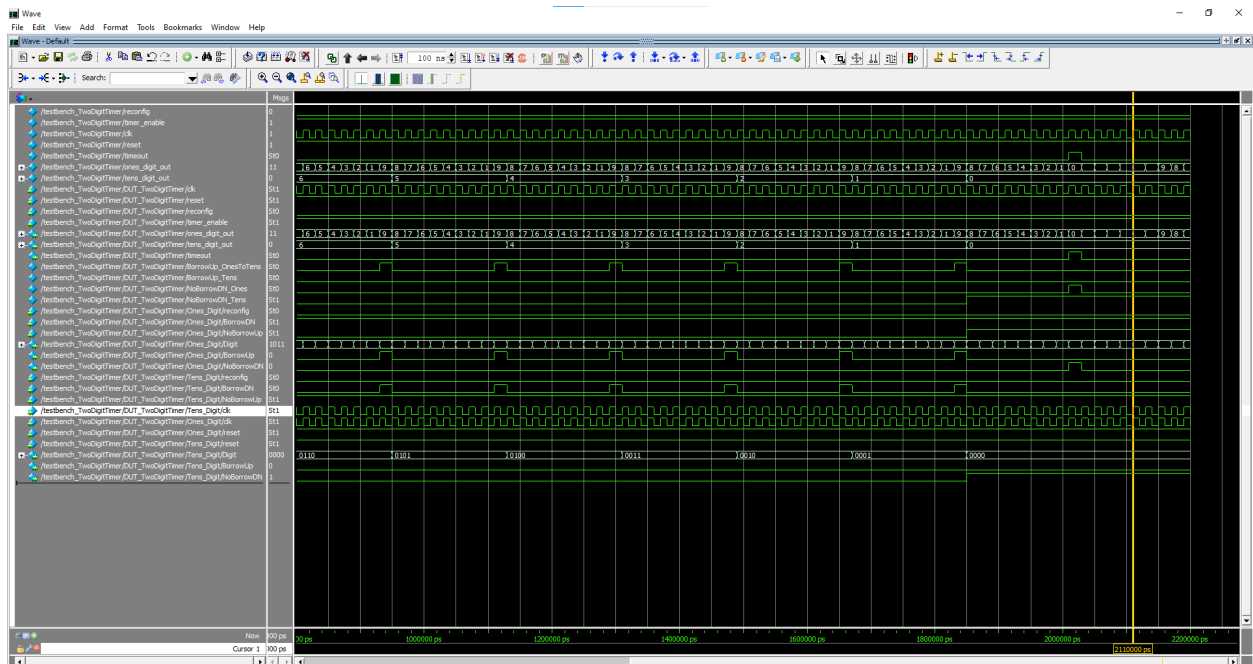
3.7. 1 Digit Timer



3.7.2 Two Digit Timer



Two digit timer simulation



Two digit timer simulation

3.8. Random Number Generator



Random number generation simulation

4. FPGA Board Testing Results

Step 1: FPGA is powered and in default settings showing 0 on all displays. The leftmost LED is lit depicting non-matching condition while the next-to-rightmost LED is lit indicating logged out status.

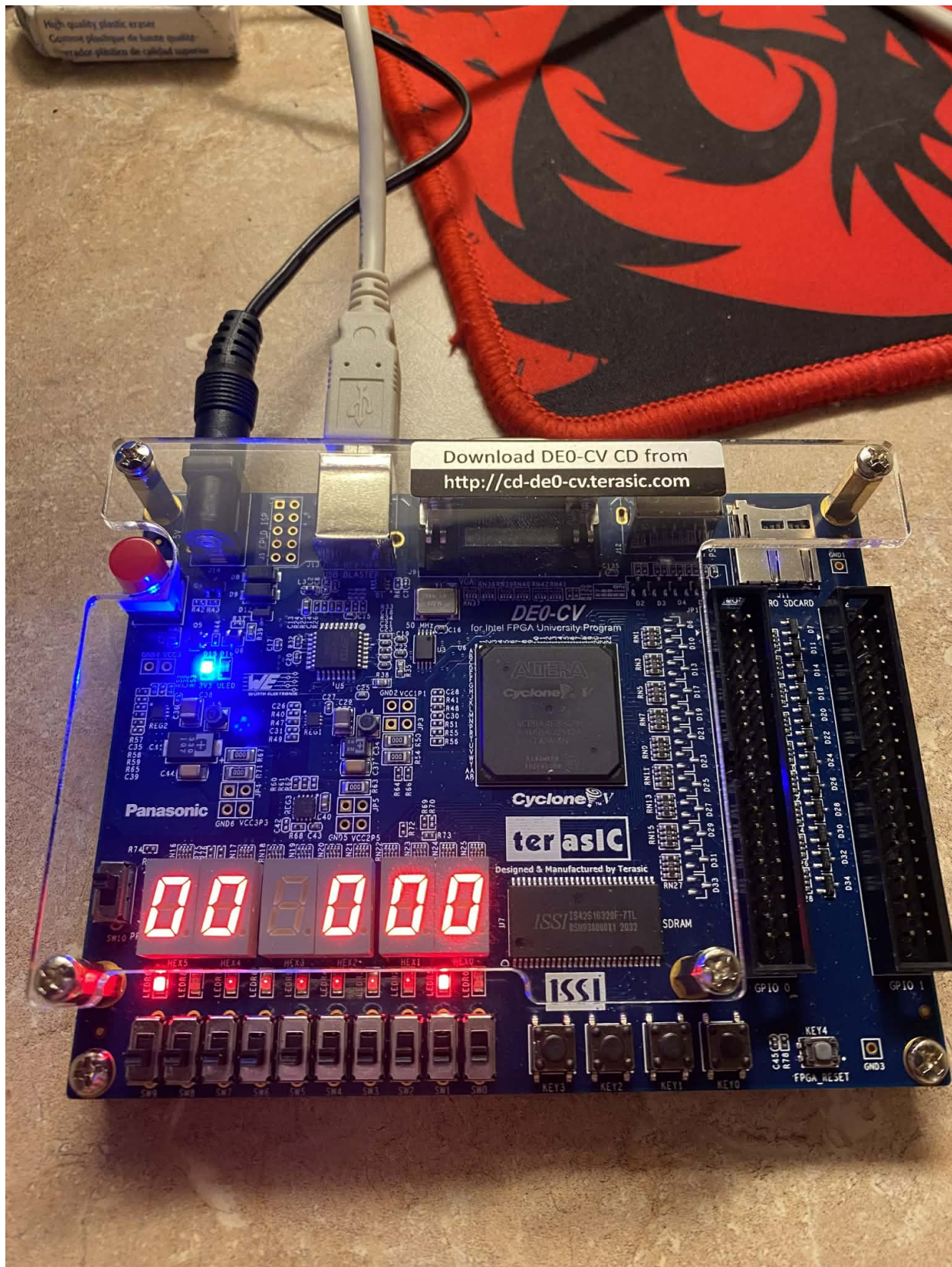


Figure 5: FPGA default settings

Step 2: After correctly entering all four password digits, timer will display 99 and rightmost LED is lit meaning it's logged in.

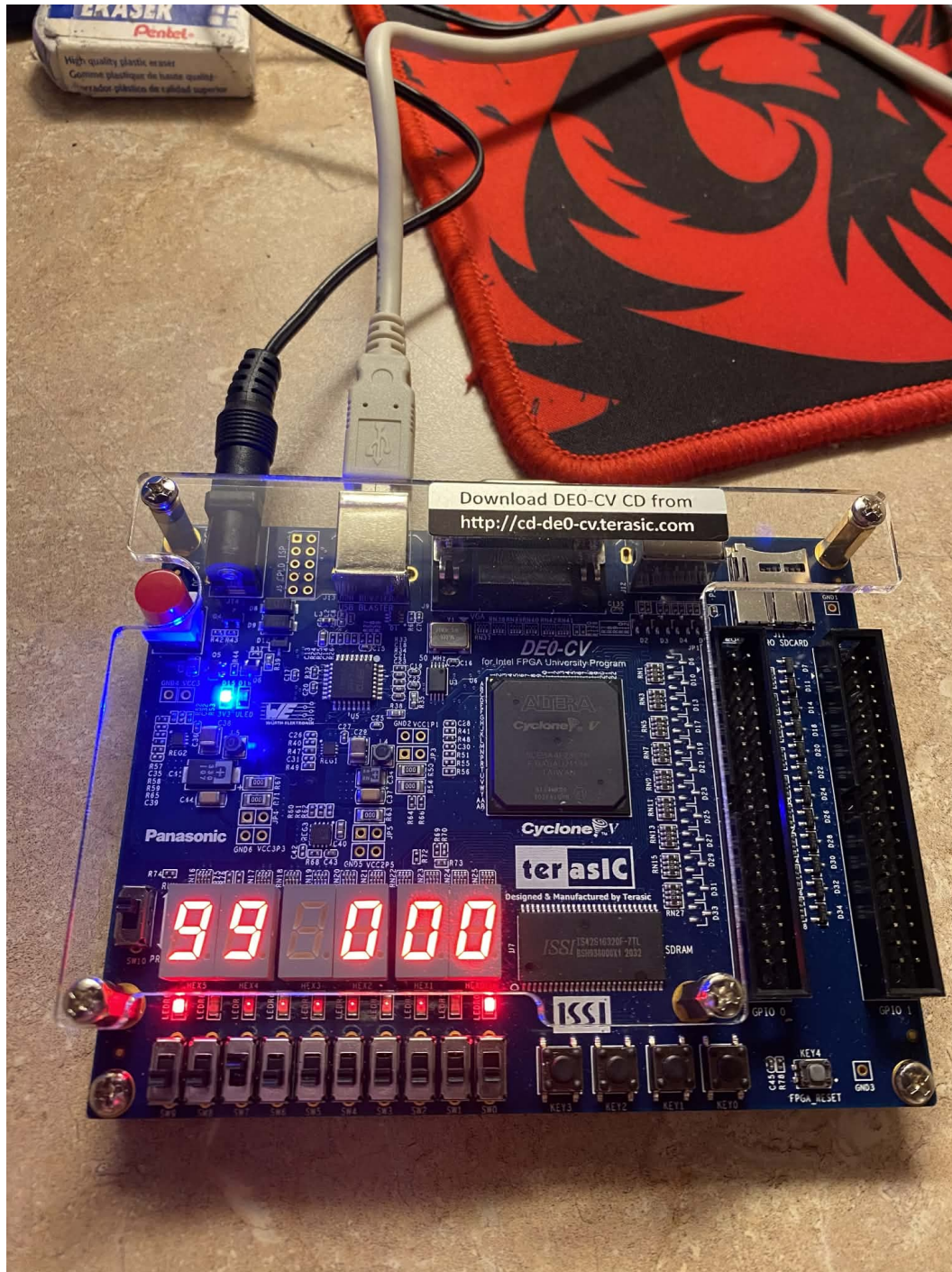


Figure 6: Status after succesfully logged in

Step 3: Pressing the Game button will have the timer start counting down.

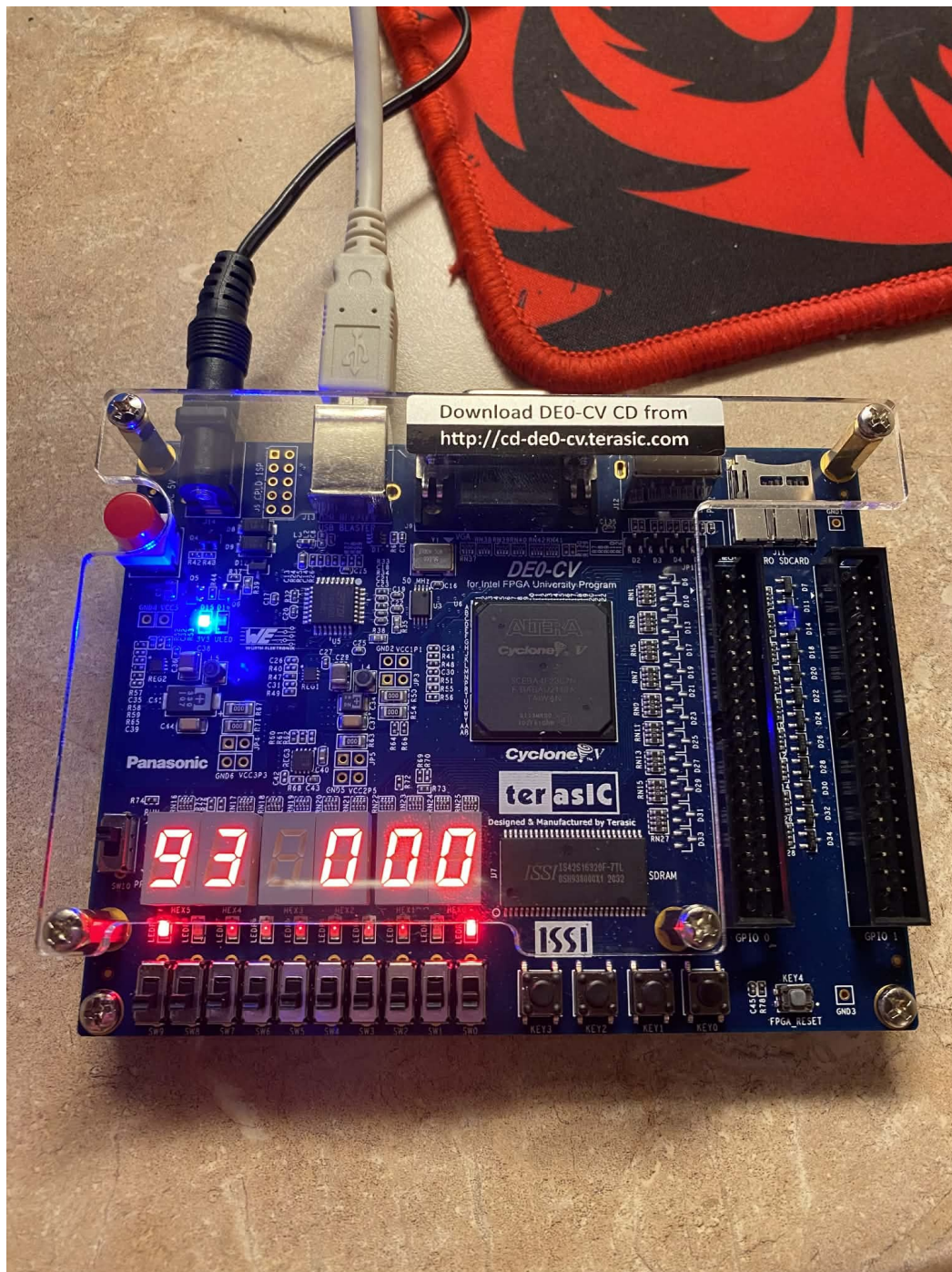


Figure 7: Timer starts counting

Step 4: Pressing the random number generation button will display a random number on the next-to-rightmost display for Player to match.

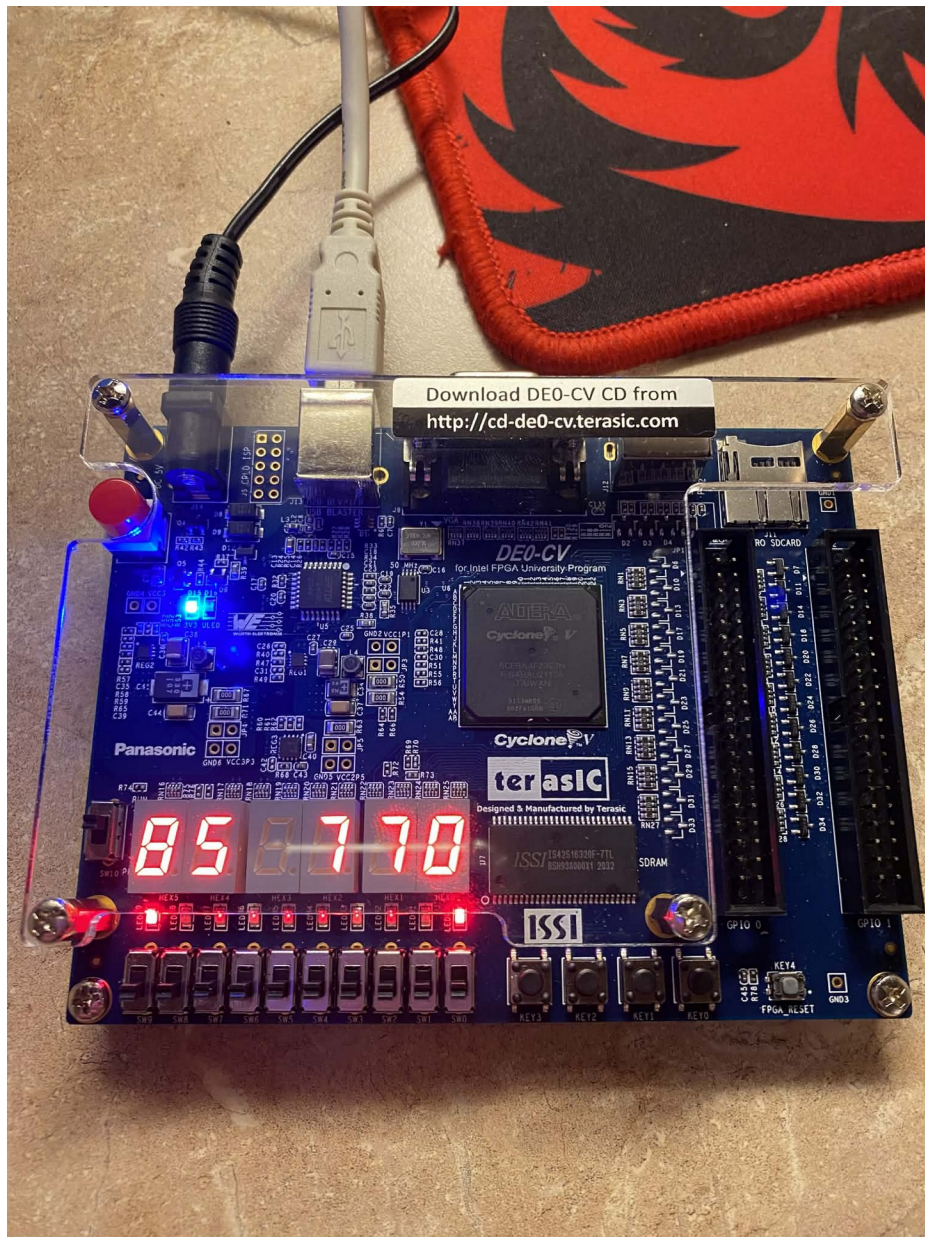


Figure 8: Random number is displayed

Step 5: Player enters a matching number (8) and pushes the LOAD button to load their number. The third-from-rightmost display shows the sum of two numbers (F means a matching pair is found). Next-to-leftmost LED is lit indicating matching condition.

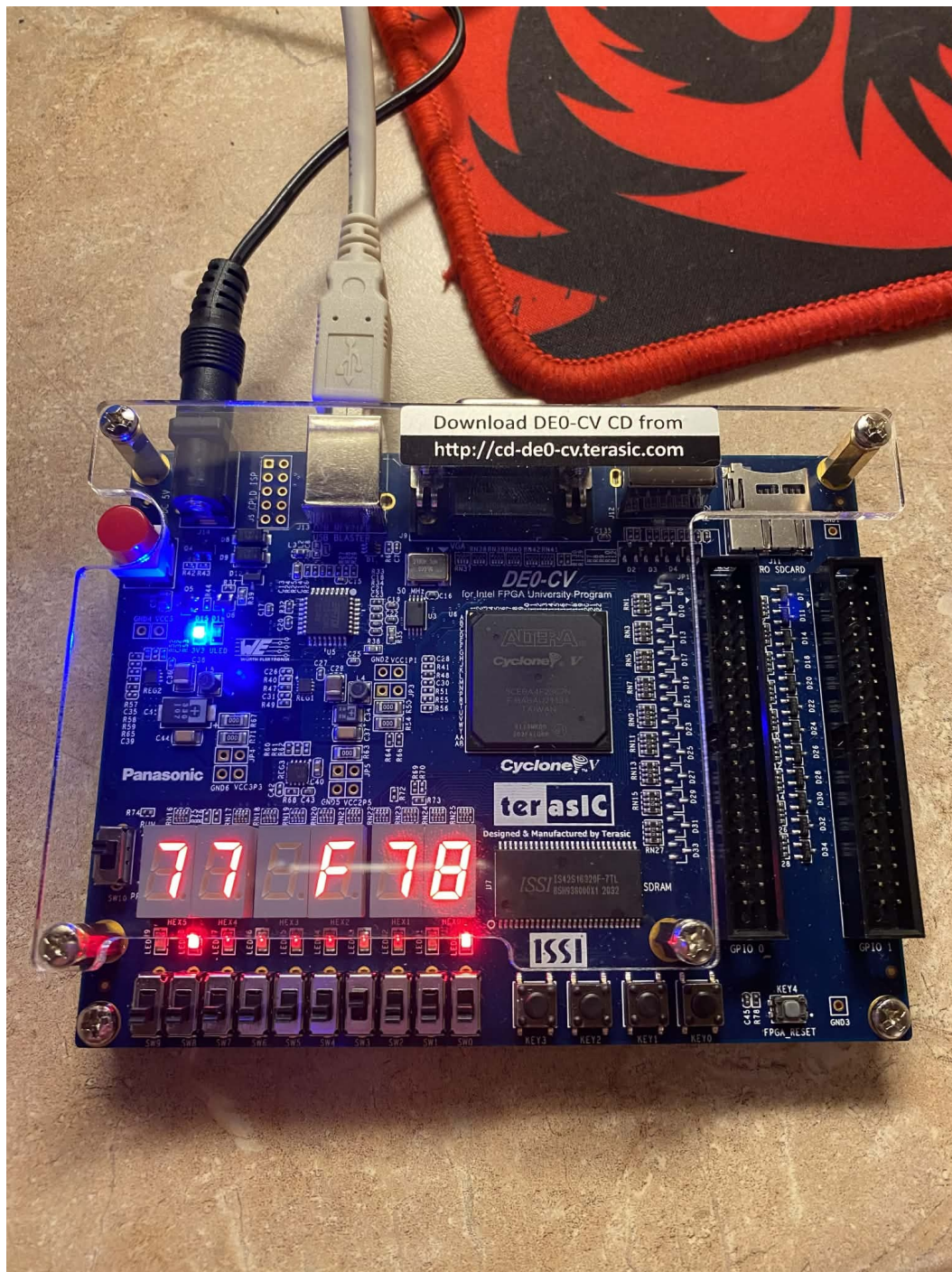


Figure 9: Matching number is found and LED is lit

Another matching pair is found. Timer still ticking down.

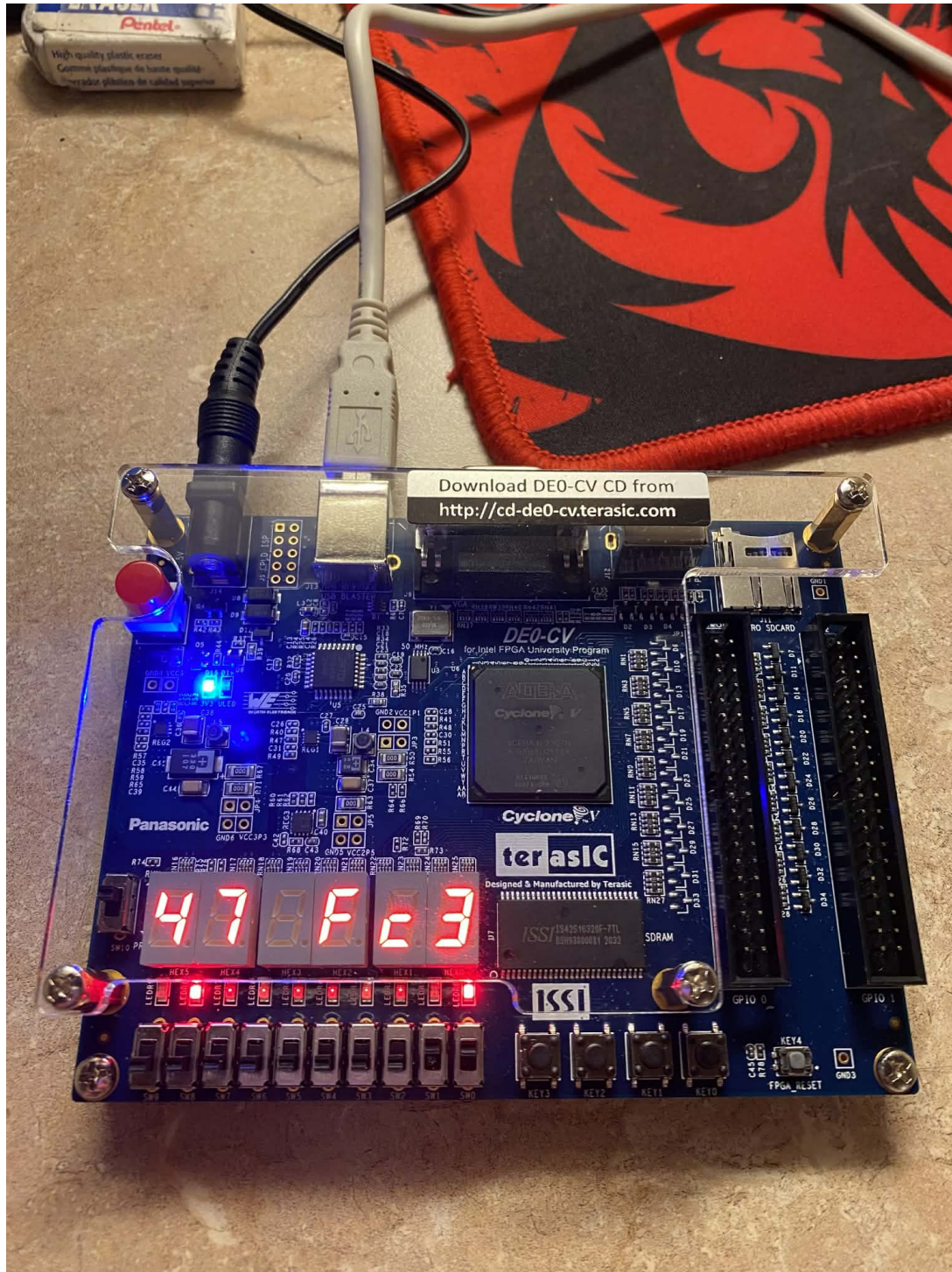


Figure 10: Another matching pair is found

Step 7: When timer hits 00, gameplay is disabled.

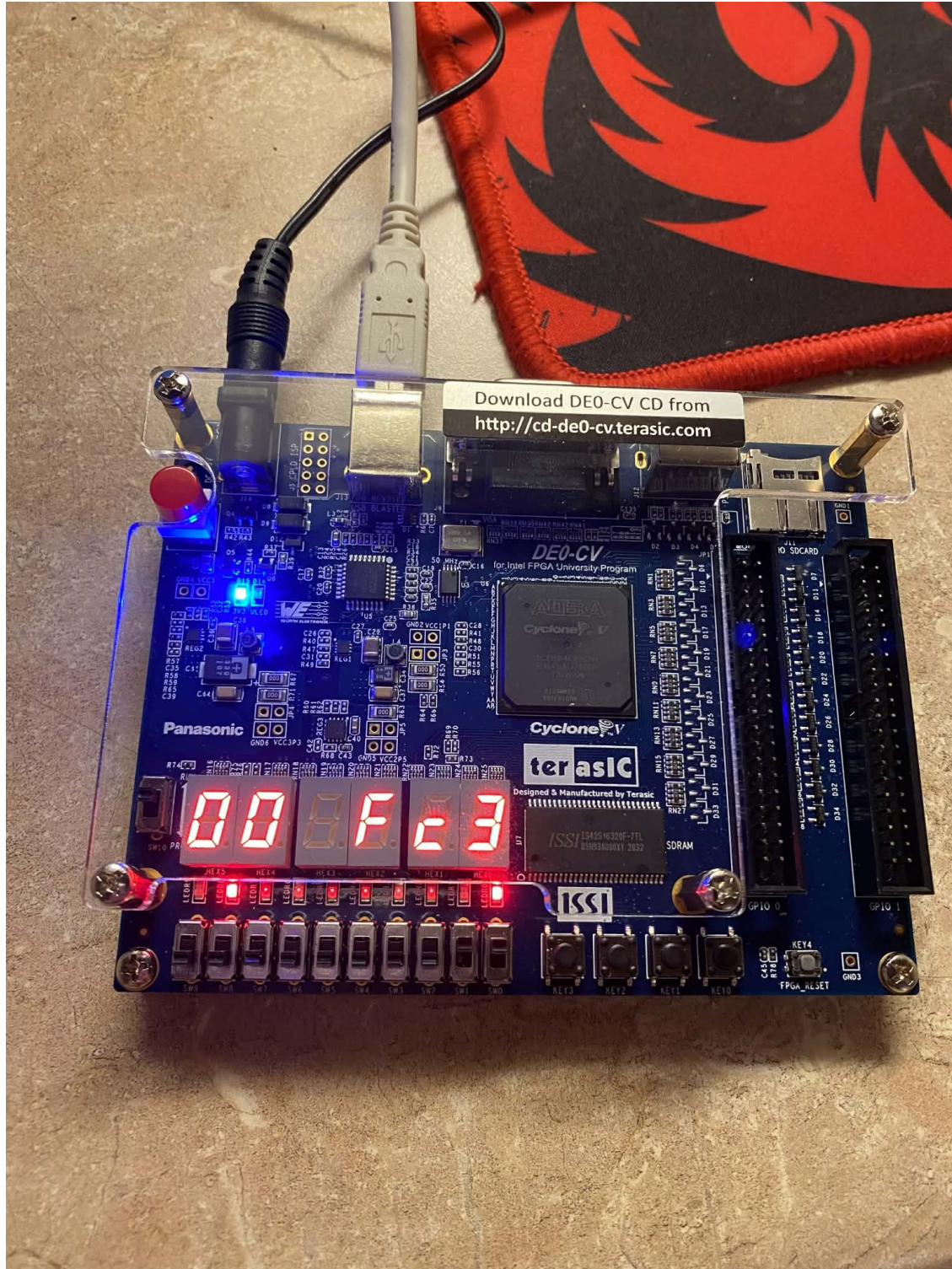


Figure 11: Game over status

Step 8: Pressing the Game/GameRestart button again will restart the timer. Pressing that same button again will start the timer and Player can start playing again.

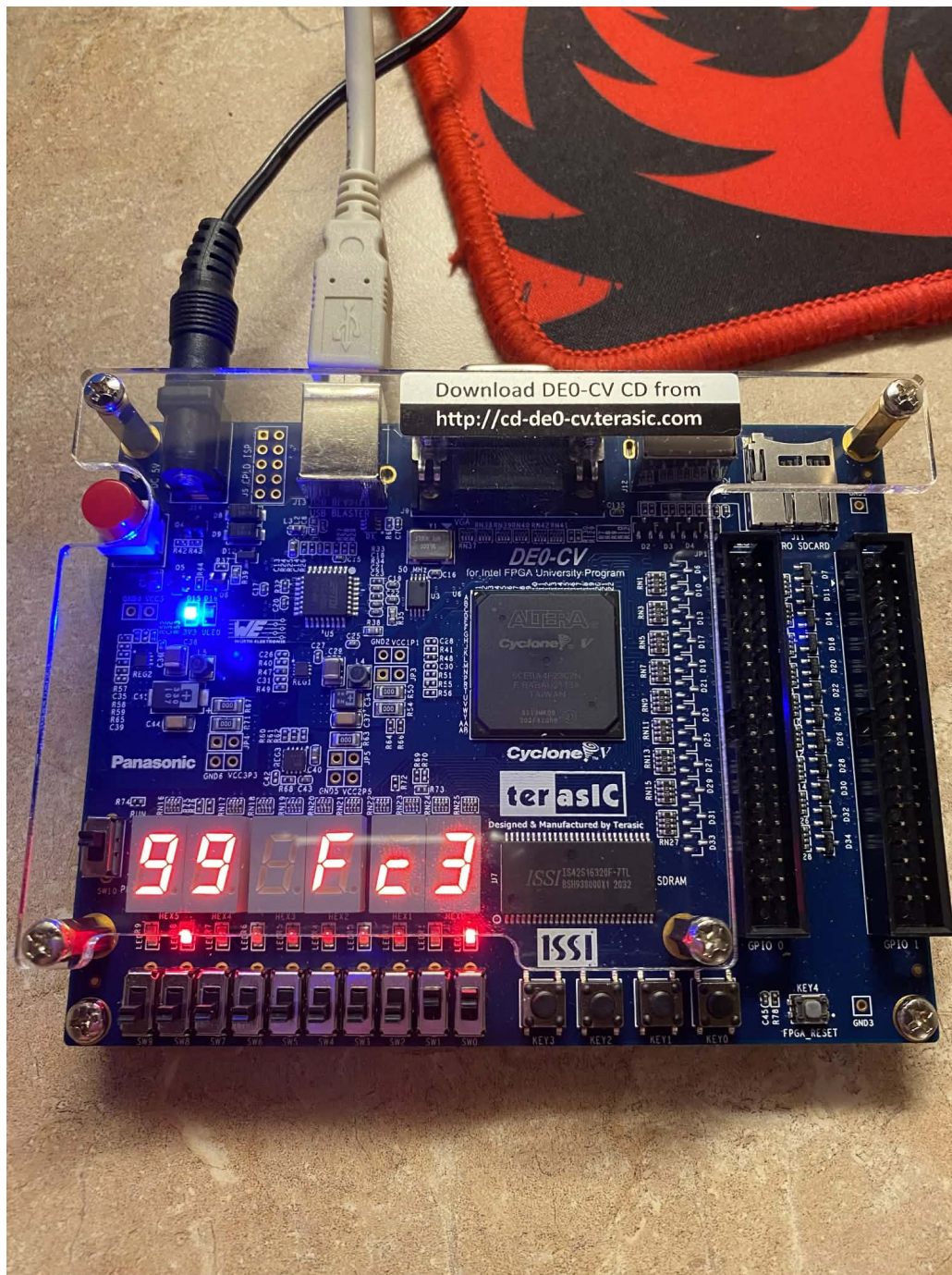


Figure 12: Timer is reloaded and game is restarted

5. Video Demo

<https://drive.google.com/file/d/13WotntIo1r8vtgiUGKsn1dgvI1ABdnqX/view?usp=sharing>

6. Conclusion

Despite having a small issue of the ones digit of the timer rolling over to 9 from 1, this project was near successfully designed, tested, and implemented on the DE0-CV FPGA board. The bonus feature of Lab 1 of lighting respective LEDs to indicate matching or non-matching conditions are also maintained. However, the new bonus feature of score tracking function wasn't implemented in this lab.

7. Appendix

```

module Lab3_NGUYEN_Tru(PNum, Pwd_Digit, B_in_P, B_in_Pwd_G,
RNG_Button, clk, reset, Decoder_out_P, D_RNG_out, D_Sum_out,
D_ones_out, D_tens_out, M_LED_top, NM_LED_top, LoggedIn,
LoggedOut);
    input [3:0] PNum, Pwd_Digit;
    input B_in_P, B_in_Pwd_G, RNG_Button, clk, reset;
    output [6:0] Decoder_out_P, D_RNG_out, D_Sum_out, D_ones_out,
D_tens_out;
    output M_LED_top, NM_LED_top, LoggedIn, LoggedOut;

    wire [3:0] Sum, P_LR_out, RNG_digit_out, tens_digit,
ones_digit;
    wire Pwd_GameStart_Restart_Shaped, Load_P_in, Load_P_out,
RNG_Gen_out, timer_reconfig, timer_enable, timeout;

    SevenSegDecoder P_Decoder (P_LR_out, Decoder_out_P);
    SevenSegDecoder Sum_Decoder (Sum, D_Sum_out);
    SevenSegDecoder RNG_Decoder (RNG_digit_out, D_RNG_out);
    SevenSegDecoder ones_Decoder (ones_digit, D_ones_out);
    SevenSegDecoder tens_Decoder (tens_digit, D_tens_out);

    FourBitAdder FourBit_Adder (P_LR_out, RNG_digit_out, Sum);

    LED_OnOff LED_Check (Sum, M_LED_top, NM_LED_top);

    ButtonShaper P_ButtonShaper (B_in_P, Load_P_in, clk, reset);
    ButtonShaper Pwd_Game_BS (B_in_Pwd_G,
Pwd_GameStart_Restart_Shaped, clk, reset);

    LoadRegister P_LoadReg (PNum, P_LR_out, clk, reset,
Load_P_out);

    TwoDigitTimer Two_digit_timer (timer_reconfig, timer_enable,
clk, reset, timeout, ones_digit, tens_digit);

    RNG RNG_top (RNG_Gen_out, clk, reset, RNG_digit_out);

    AccessController Access_Controller
(Pwd_GameStart_Restart_Shaped, Pwd_Digit, Load_P_in, RNG_Button,
timeout, clk, reset, LoggedIn, LoggedOut, Load_P_out,
RNG_Gen_out, GoodUntilNow, timer_reconfig, timer_enable);

endmodule

```

Figure 17: Top-level module Lab3_NGUYEN_Tru


```
D:/work/ADD_Demos/Lab2_NGUYEN_Tru/sim_bs/testbench_ButtonShaper.v
File Edit View Tools Bookmarks Window Help
D:/work/ADD_Demos/Lab2_NGUYEN_Tru/sim_bs/testbench_ButtonShaper.v - Default

Ln#
1 // ECE 6370
2 // Author: Tru Nguyen
3 // Module name: testbench_ButtonShaper
4 // Function: a testbench to observe waveforms and verify the design of ButtonShaper module
5
6 `timescale 1 ns/100 ps
7
8 module testbench_ButtonShaper();
9     reg B_in, clk, reset;
10    wire B_out;
11
12    always                // artificial clk
13    begin
14        clk = 1'b0;
15        #10;
16        clk = 1'b1;
17        #10; // every cycle is 20ns long: 10ns high & 10ns low
18    end
19
20    ButtonShaper DUT_ButtonShaper(B_in, B_out, clk, reset);
21
22    initial
23    begin
24        B_in = 1'b1; // button is not pressed initially
25        reset = 1'b1; // reset button is not pressed
26
27        @ (posedge clk);
28        @ (posedge clk);
29        #5 reset = 1'b0;
30        @ (posedge clk);
31        @ (posedge clk);
32        @ (posedge clk);
33        @ (posedge clk);
34        #5 reset = 1'b1;
35        @ (posedge clk);
36        @ (posedge clk);
37        #5 B_in = 1'b0;
38        @ (posedge clk);
39        @ (posedge clk);
40        @ (posedge clk);
41        @ (posedge clk);
42        @ (posedge clk);
43        @ (posedge clk);
44        @ (posedge clk);
45        @ (posedge clk);
46        @ (posedge clk);
47        @ (posedge clk);
48        #5 B_in = 1'b1;
49
50        @ (posedge clk);
51        @ (posedge clk);
52        @ (posedge clk);
53        @ (posedge clk);
54        @ (posedge clk);
55        @ (posedge clk);
56        @ (posedge clk);
57        @ (posedge clk);
58        #5 reset = 1'b0;
59        @ (posedge clk);
```

Figure 18: The button shaper testbench module part 1

```

49
50     @(posedge clk);
51     @(posedge clk);
52     @(posedge clk);
53     @(posedge clk);
54     @(posedge clk);
55     @(posedge clk);
56     @(posedge clk);
57     @(posedge clk);
58     #5 reset = 1'b0;
59     @(posedge clk);
60     @(posedge clk);
61     @(posedge clk);
62     @(posedge clk);
63     #5 reset = 1'b1;
64     @(posedge clk);
65     @(posedge clk);
66     #5 B_in = 1'b0;
67     @(posedge clk);
68     @(posedge clk);
69     @(posedge clk);
70     @(posedge clk);
71     @(posedge clk);
72     @(posedge clk);
73     @(posedge clk);
74     @(posedge clk);
75     @(posedge clk);
76     @(posedge clk);
77     #5 B_in = 1'b1;
78
79
80
81     end
82
83     endmodule
84

```

Figure 19: The button shaper testbench module part 2

D:/work/ADD_Demos/Lab2_NGUYEN_Tru/sim_bs/ButtonShaper.v

File Edit View Tools Bookmarks Window Help

D:/work/ADD_Demos/Lab2_NGUYEN_Tru/sim_bs/ButtonShaper.v - Default

```
Ln#
1 // ECE 6370
2 // Author: Tru Nguyen
3 // Module name: ButtonShaper
4 // Function: generates a clean one-clock-cycle pulse from raw button push signal
5
6 module ButtonShaper(B_in, B_out, clk, reset);
7     input B_in, clk, reset;
8     output B_out;
9     reg B_out;
10    parameter INIT = 0, PULSE = 1, WAIT = 2;
11    reg [1:0] State, NextState;
12
13    always @(State, B_in) begin
14        case(State)
15            INIT: begin
16                B_out = 1'b0;
17                if (B_in == 1'b0)
18                    NextState = PULSE;
19                else
20                    NextState = INIT;
21            end
22
23            PULSE: begin
24                B_out = 1'b1;
25                NextState = WAIT;
26            end
27
28            WAIT: begin
29                B_out = 1'b0;
30                if (B_in == 1'b0)
31                    NextState = WAIT;
32                else
33                    NextState = INIT;
34            end
35        endcase
36    end
37
38    always @(posedge clk) begin
39        if (reset == 1'b0)
40            State <= INIT;
41        else
42            State <= NextState;
43        end
44    end
45 endmodule
46
```

Figure 20: The button shaper module

D:/work/ADD_Demos/Lab2_NGUYEN_Tru/sim_loadreg/LoadRegister.v - Default

```
Ln#
1 module LoadRegister(D_in, D_out, clk, reset, Load);
2   input [3:0] D_in;
3   output [3:0] D_out;
4   input clk, reset, Load;
5   reg [3:0] D_out;
6
7   always@(posedge clk) // sequential logic "always" blocks only include clk or sometimes reset in the sensitivity list
8   begin
9       if (reset == 1'b0)
10      begin
11          D_out <= 4'b0000;
12      end
13      else
14      begin
15          if (Load == 1'b1)
16          begin
17              D_out <= D_in;
18          end
19      end // "else" statement in an "if" statement is optional in sequential logic
20  end
21 endmodule
22
```

Figure 21: The load register module

```

D:/work/ADD_Demos/Lab2_NGUYEN_Tru/sim_loadreg/testbench_LoadRegister.v - Default
Ln#
1 // ECE 6370
2 // Author: Tru Nguyen
3 // Module name: testbench_LoadRegister
4 // Function: a testbench to observe signal waveforms and verify the design of LoadRegister module
5
6 `timescale 1 ns/100 ps
7
8 module testbench_LoadRegister();
9     reg clk, reset, Load;
10    reg [3:0] D_in;
11    wire [3:0] D_out;
12
13    always                // artificial clk
14    begin
15        clk = 1'b0;
16        #10;
17        clk = 1'b1;
18        #10; // every cycle is 20ns long: 10ns high & 10ns low
19    end
20
21    LoadRegister DUT_LoadRegister(D_in, D_out, clk, reset, Load);
22
23    initial
24    begin
25        reset = 1'b1; // button is not pressed at begin (reset = 0 means pressed; reset = 1 means not pressed)
26        Load = 1'b0;
27        D_in = 4'b0000;
28
29        @(posedge clk); // skip the first clk rising edge
30        @(posedge clk); // skip the second clk rising edge
31        #5 reset = 1'b0; // pull reset low 5 ns after the second clk rising edge/button pressed
32        @(posedge clk);
33        @(posedge clk);
34        @(posedge clk);
35        @(posedge clk);
36        #5 reset = 1'b1; // pull reset high after 4 and a half clk rising edge/button released
37        @(posedge clk);
38        @(posedge clk);
39        @(posedge clk);
40        @(posedge clk);
41        D_in = 4'b1010;
42        @(posedge clk);
43        Load = 1'b1; // pull Load high meaning D_out is rdy for data in D_in to be loaded in
44        @(posedge clk);
45        Load = 1'b0; // pull Load low after 1 clk rising edge
46        @(posedge clk);
47        D_in = 4'b0110;
48        @(posedge clk);
49        @(posedge clk);
50        D_in = 4'b1110;
51        Load = 1'b1;
52        @(posedge clk);
53        Load = 1'b0;
54        @(posedge clk);
55    end
56
57 endmodule
58

```

Figure 22: The load register testbench module

```

// ECE 6370
// Author: Tru Nguyen, 1032
// Module: A design of a 7-segment decoder
// This module takes one 4-bit binary input signal decoder_in
// and produces a 7-bit output signal decoder_out that
// displays the corresponding input value in hexadecimal on the
FPGA

module SevenSegDecoder(decoder_in, decoder_out);
    input [3:0] decoder_in;
    output [6:0] decoder_out;
    reg [6:0] decoder_out;

    always @(decoder_in)
    begin
        case(decoder_in)
            4'b0000: begin decoder_out = 7'b1000000; end
            4'b0001: begin decoder_out = 7'b1111001; end
            4'b0010: begin decoder_out = 7'b0100100; end
            4'b0011: begin decoder_out = 7'b0110000; end
            4'b0100: begin decoder_out = 7'b0011001; end
            4'b0101: begin decoder_out = 7'b0010010; end
            4'b0110: begin decoder_out = 7'b0000010; end
            4'b0111: begin decoder_out = 7'b1111000; end
            4'b1000: begin decoder_out = 7'b0000000; end
            4'b1001: begin decoder_out = 7'b0010000; end
            4'b1010: begin decoder_out = 7'b0001000; end
            4'b1011: begin decoder_out = 7'b0000011; end
            4'b1100: begin decoder_out = 7'b0100111; end
            4'b1101: begin decoder_out = 7'b0100001; end
            4'b1110: begin decoder_out = 7'b0000110; end
            4'b1111: begin decoder_out = 7'b0001110; end
            default: begin decoder_out = 7'b1111111; end
        endcase
    end

endmodule

```

Figure 30: The 7-segment decoder module


```

`timescale 1 ns/100 ps
module testbench_SevenSegDecoder();

    reg [3:0] decoder_in_s;
    wire [6:0] decoder_out_s;

    SevenSegDecoder DUT_SevenSegDecoder(decoder_in_s,
decoder_out_s);

    initial
        begin
            decoder_in_s = 4'b0000;
            #10 decoder_in_s = 4'b0001;
            #10 decoder_in_s = 4'b0010;
            #10 decoder_in_s = 4'b0011;
            #10 decoder_in_s = 4'b0100;
            #10 decoder_in_s = 4'b0101;
            #10 decoder_in_s = 4'b0110;
            #10 decoder_in_s = 4'b0111;
            #10 decoder_in_s = 4'b1000;
            #10 decoder_in_s = 4'b1001;
            #10 decoder_in_s = 4'b1010;
            #10 decoder_in_s = 4'b1011;
            #10 decoder_in_s = 4'b1100;
            #10 decoder_in_s = 4'b1101;
            #10 decoder_in_s = 4'b1110;
            #10 decoder_in_s = 4'b1111;
            end
endmodule

```

Figure 31: The 7-segment decoder testbench module

```

// ECE 6370
// Author: Tru Nguyen, 1032
// Module: A design of a 4-bit binary adder
// This module takes two 4-bit binary input signals P1Num and
// P2Num, calculating the sum of the input signals, and store
// the 4-bit binary result to the output signal Sum

module FourBitAdder(P1Num, P2Num, Sum);
    input [3:0] P1Num, P2Num;
    output [3:0] Sum;
    reg [3:0] Sum;

    always @(P1Num, P2Num)
        begin
            Sum = P1Num + P2Num;
        end
endmodule

```

Figure 32: The 4-bit adder module

```

`timescale 1 ns/100 ps

module testbench_FourBitAdder();

    reg [3:0] P1Num_s, P2Num_s;
    wire [3:0] Sum_s;

    FourBitAdder DUT_FourBitAdder(P1Num_s, P2Num_s, Sum_s);

    initial
        begin
            P1Num_s = 4'b1010; P2Num_s = 4'b0101; // matching case:
10+5=15
            #10 P1Num_s = 4'hC; P2Num_s = 4'h1; // non-matching case:
12+1=13
            #10 P1Num_s = 4'd11; P2Num_s = 4'd6; // overflow: 11+6=
17 -> (1) 0001

        end
endmodule

```

Figure 33: The 4-bit adder testbench module

```

// ECE 6370
// Author: Tru Nguyen, 1032
// Module: A verification design to check if the sum of two 4-
bit binary numbers equals F
// This module takes one 4-bit binary input signal Sum, checks
if Sum equals F, if so, light up the left-most LED, otherwise
light up the right-most LED.

module LED_OnOff(Sum, M_LED, NM_LED);
    input [3:0] Sum;
    output M_LED, NM_LED;
    reg M_LED, NM_LED;

    always @(Sum, M_LED, NM_LED)
        begin
            if (Sum == 4'b1111)
                begin
                    M_LED = 1'b1;
                end
            else
                begin
                    NM_LED = 1'b1;
                end
        end
    end
endmodule

```

Figure 34: The verification module of matching and non-matching conditions


```

// ECE 6370
// Author: Tru Nguyen - 1032
// Module name: AccessController
// Function: Implement a 4-digit password-based login system
utilizing the 1-procedure FSM method

module AccessController(Pwd_GameStart_Restart_Shaped,
Pwd_And_GamePlay_Digit, Load_P_in, RNG_Button, timeout, clk,
reset, LoggedIn, LoggedOut, Load_P_out, RNG_Gen_out,
GoodUntilNow, timer_reconfig, timer_enable);
    input Pwd_GameStart_Restart_Shaped, Load_P_in, RNG_Button,
clk, reset, timeout;
    input [3:0] Pwd_And_GamePlay_Digit;
    output reg LoggedIn, LoggedOut, Load_P_out, RNG_Gen_out,
GoodUntilNow, timer_reconfig, timer_enable;

    parameter DIGIT1 = 0, DIGIT2 = 1, DIGIT3 = 2, DIGIT4 = 3,
VERIFY = 4, RECONFIGTIMER = 5, WAITFORGAMESTART = 6, GAMEPLAY =
7, GAMEOVER = 8;

    reg [3:0] State;

    always @(posedge clk) begin
        if (reset == 1'b0) begin
            LoggedIn <= 1'b0;
            LoggedOut <= 1'b1;
            Load_P_out <= 1'b0;
            RNG_Gen_out <= 1'b1;
            GoodUntilNow <= 1'b1;
            State <= DIGIT1;
        end

        else begin
            case(State)
                DIGIT1: begin
                    LoggedIn <= 1'b0;
                    LoggedOut <= 1'b1;
                    Load_P_out <= 1'b0;
                    RNG_Gen_out <= 1'b1;
                    GoodUntilNow <= 1'b1;
                    timer_reconfig <= 1'b0;
                    timer_enable <= 1'b0;

                    if (Pwd_GameStart_Restart_Shaped == 1'b1) begin
                        if (Pwd_And_GamePlay_Digit == 4'b0001) begin
// check if first digit is 1 (last 4 digits of ID = 1032)
// correct digit
// DO NOT TOUCH GoodUntilNow
                        end

```

```

        // correct digit
        // DO NOT TOUCH GoodUntilNow
    end
    else begin
        GoodUntilNow <= 1'b0; // wrong digit
    end
    State <= DIGIT2;
end
else begin
    State <= DIGIT1;
end

end

DIGIT2: begin
    LoggedIn <= 1'b0;
    LoggedOut <= 1'b1;
    Load_P_out <= 1'b0;
    RNG_Gen_out <= 1'b1;
    timer_reconfig <= 1'b0;
    timer_enable <= 1'b0;

    if (Pwd_GameStart_Restart_Shaped == 1'b1) begin
        if (Pwd_And_GamePlay_Digit == 4'b0000) begin
            // check if second digit is 0 (last 4 digits of ID = 1032)
            // correct digit
            // DO NOT TOUCH GoodUntilNow
        end
        else begin
            GoodUntilNow <= 1'b0; // wrong digit
        end
        State <= DIGIT3;
    end
    else begin
        State <= DIGIT2;
    end

end

DIGIT3: begin
    LoggedIn <= 1'b0;
    LoggedOut <= 1'b1;
    Load_P_out <= 1'b0;
    RNG_Gen_out <= 1'b1;
    timer_reconfig <= 1'b0;
    timer_enable <= 1'b0;

    if (Pwd_GameStart_Restart_Shaped == 1'b1) begin
        if (Pwd_And_GamePlay_Digit == 4'b0011) begin

```

```

-
    if (Pwd_GameStart_Restart_Shaped == 1'b1) begin
        if (Pwd_And_GamePlay_Digit == 4'b0011) begin
            // check if third digit is 3 (last 4 digits of ID = 1032)
            // correct digit
            // DO NOT TOUCH GoodUntilNow
        end
        else begin
            GoodUntilNow <= 1'b0; // wrong digit
        end
        State <= DIGIT4;
    end
    else begin
        State <= DIGIT3;
    end
end

    end

    DIGIT4: begin
        LoggedIn <= 1'b0;
        LoggedOut <= 1'b1;
        Load_P_out <= 1'b0;
        RNG_Gen_out <= 1'b1;
        timer_reconfig <= 1'b0;
        timer_enable <= 1'b0;

        if (Pwd_GameStart_Restart_Shaped == 1'b1) begin
            if (Pwd_And_GamePlay_Digit == 4'b0010) begin
                // check if last digit is 2 (last 4 digits of ID = 1032)
                // correct digit
                // DO NOT TOUCH GoodUntilNow
            end
            else begin
                GoodUntilNow <= 1'b0; // wrong digit
            end
            State <= VERIFY;
        end
        else begin
            State <= DIGIT4;
        end
    end

    end

    VERIFY: begin
        LoggedIn <= 1'b0;
        LoggedOut <= 1'b1;
        Load_P_out <= 1'b0;
        RNG_Gen_out <= 1'b1;
        timer_reconfig <= 1'b0;

```

```

        RNG_Gen_out <= 1'b1;
        timer_reconfig <= 1'b0;
        timer_enable <= 1'b0;

        if (GoodUntilNow == 1'b1) begin
            State <= RECONFIGTIMER; // all four digits are
correct
        end
        else begin
            State <= DIGIT1; // at least one digit is wrong, reset
the password entry
        end

        end

        RECONFIGTIMER: begin // loggedin and display 99 on timer
but still block all input access
        LoggedIn <= 1'b1;
        LoggedOut <= 1'b0;
        Load_P_out <= 1'b0;
        RNG_Gen_out <= 1'b1;
        timer_reconfig <= 1'b1;
        timer_enable <= 1'b0;

        State <= WAITFORGAMESTART;

end

WAITFORGAMESTART: begin
    LoggedIn <= 1'b1;
    LoggedOut <= 1'b0;
    Load_P_out <= 1'b0;
    RNG_Gen_out <= 1'b1;
    timer_reconfig <= 1'b0;

    if (Pwd_GameStart_Restart_Shaped == 1'b1) begin
        timer_enable <= 1'b1;
        State <= GAMEPLAY;
    end

    else begin
        timer_enable <= 1'b0;
        State <= WAITFORGAMESTART;
    end

end

GAMEPLAY: begin

```

```

GAMEPLAY: begin
    LoggedIn <= 1'b1;
    LoggedOut <= 1'b0;
    Load_P_out <= Load_P_in;
    RNG_Gen_out <= RNG_Button;
    timer_enable <= 1'b1;
    timer_reconfig <= 1'b0;

    if (timeout == 1'b1) begin
        State <= GAMEOVER;
    end

    else begin
        State <= GAMEPLAY;
    end

end

GAMEOVER: begin
    LoggedIn <= 1'b1;
    LoggedOut <= 1'b0;
    Load_P_out <= 1'b0;
    RNG_Gen_out <= 1'b1;
    timer_reconfig <= 1'b0;
    timer_enable <= 1'b0;

    if (Pwd_GameStart_Restart_Shaped == 1'b1) begin
        State <= RECONFIGTIMER;
    end

    else begin
        State <= GAMEOVER;
    end

end

default: begin // DIGIT1 state
    LoggedIn <= 1'b0;
    LoggedOut <= 1'b1;
    Load_P_out <= 1'b0;
    RNG_Gen_out <= 1'b1;
    GoodUntilNow <= 1'b1;

    if (Pwd_GameStart_Restart_Shaped == 1'b1) begin
        if (Pwd_And_GamePlay_Digit == 4'b0001) begin
            check if first digit is 1 (last 4 digits of ID = 1032)
            // correct digit
            // DO NOT TOUCH GoodUntilNow
        end
        else begin
            GoodUntilNow <= 1'b0; // wrong digit
        end
        State <= DIGIT2;
    end
    else begin
        State <= DIGIT1;
    end
end

```

```

        if (Pwd_GameStart_Restart_Shaped == 1'b1) begin
            if (Pwd_And_GamePlay_Digit == 4'b0001) begin //
check if first digit is 1 (last 4 digits of ID = 1032)
                // correct digit
                // DO NOT TOUCH GoodUntilNow
            end
            else begin
                GoodUntilNow <= 1'b0; // wrong digit
            end
            State <= DIGIT2;
        end
        else begin
            State <= DIGIT1;
        end
    end

endcase
end

end

endmodule

```

```

module Counter(Count, clk, reset, Count_out);
    input Count, clk, reset;
    output reg [3:0] Count_out;

    always @(posedge clk) begin
        if (reset == 1'b0) begin
            Count_out <= 4'd0;
        end

        else begin
            if (Count == 1'b1) begin
                Count_out <= Count_out+1;
            end
        end
    end
end
endmodule

```

```

// ECE 6370
// Author: Tru Nguyen
// Module name: countTo10
// Function: counts 100 ms timeout pulses and generates a 1
second timeout signal. Once the count limit is reached, it
resets and asserts OneSecTimeOut for one clock cycle.
// (Note: Counter size is reduced for simulation purposes.)

module countTo10(HundredMsTimeOut, clk, reset, OneSecTimeOut,
Count_out);
    input HundredMsTimeOut, clk, reset;
    output reg OneSecTimeOut;
    output reg [3:0] Count_out;
    //output [1:0] Count_out; // for sim only
    //reg OneSecTimeOut;
    //reg [3:0] Count_out;
    //reg [1:0] Count_out; // for sim only

    always @(posedge clk) begin
        if (reset == 1'b0) begin
            Count_out <= 4'd0;
            OneSecTimeOut <= 1'b0;
        end
        else begin
            if (HundredMsTimeOut == 1'b1) begin
                if (Count_out == 4'd10) begin
                    Count_out <= 4'd0;
                    OneSecTimeOut <= 1'b1;
                end
            else begin
                Count_out <= Count_out+1;
                OneSecTimeOut <= 1'b0;
            end
        end
        else begin
            OneSecTimeOut <= 1'b0;
        end
    end
end

endmodule

```

```

// ECE 6370
// Author: Tru Nguyen
// Module name: countTo100
// Function: counts incoming 1 ms timeout pulses and generates a
100 ms timeout signal. When the count reaches its limit, it
resets and produces a one-cycle output pulse.
// (Note: Counter size is reduced for simulation purposes.)

module countTo100(OneMsTimeout, clk, reset, HundredMsTimeout,
Count_out);
    input OneMsTimeout, clk, reset;
    output reg HundredMsTimeout;
    output reg [6:0] Count_out;
    //output [1:0] Count_out; // for sim only
    //reg HundredMsTimeout;
    //reg [6:0] Count_out;
    //reg [1:0] Count_out; // for sim only

    always @(posedge clk) begin
        if (reset == 1'b0) begin
            Count_out <= 7'd0;
            HundredMsTimeout <= 1'b0;
        end
        else begin
            if (OneMsTimeout == 1'b1) begin
                if (Count_out == 7'd100) begin
                    Count_out <= 7'd0;
                    HundredMsTimeout <= 1'b1;
                end
            end
            else begin
                Count_out <= Count_out+1;
                HundredMsTimeout <= 1'b0;
            end
        end
        else begin
            HundredMsTimeout <= 1'b0;
        end
    end
end

endmodule

```



```

module digitTimer(reconfig, BorrowDN, NoBorrowUp, clk, reset,
Digit, BorrowUp, NoBorrowDN);
    input reconfig, BorrowDN, NoBorrowUp, clk, reset;
    output [3:0] Digit;
    output BorrowUp, NoBorrowDN;

    reg [3:0] Digit;
    reg BorrowUp, NoBorrowDN;

    always @(posedge clk) begin
        if (reset == 1'b0) begin
            Digit <= 4'd0;
            BorrowUp <= 1'b0;
            NoBorrowDN <= 1'b1;
        end

        else begin
            BorrowUp <= 1'b0;

            if (reconfig == 1'b0) begin

                if (BorrowDN == 1'b1) begin

                    if (Digit == 4'd1) begin

                        if (NoBorrowUp == 1'b0) begin
                            Digit <= 4'd9;
                            BorrowUp <= 1'b1;
                            NoBorrowDN <= 1'b0;
                        end // NoBorrowUp

                        else begin
                            BorrowUp <= 1'b0;
                            Digit <= 4'd0;
                            NoBorrowDN <= 1'b1;
                        end // NoBorrowUp else

                    end // Digit

                else begin
                    Digit <= Digit - 1;
                    NoBorrowDN <= 1'b0;
                    // BorrowUp <= 1'b0;
                    // NoBorrowDN <= 1'b0;
                end // Digit else

            end // BorrowDN

        else begin
            if (Digit == 4'd0) begin
                NoBorrowDN <= 1'b1;
            end

            else begin

```

```

        end // BorrowDN

    else begin
        if (Digit == 4'd0) begin
            NoBorrowDN <= 1'b1;
        end

        else begin
            NoBorrowDN <= 1'b0;
        end

    end // BorrowDN else

    end // reconfig

    else begin
        Digit <= 4'd9;
        BorrowUp <= 1'b0;
        NoBorrowDN <= 1'b0;

    end // reconfig else

    end // reset else
end // always
endmodule

```

```

// ECE 6370
// Author: Tru Nguyen
// Module name: OneMsTimer
// Function: generates a 1 millisecond timeout pulse by counting
clock cycles when enabled. When the internal counter reaches its
maximum value, it resets and asserts OneMsTimeOut for one clock
cycle.

module OneMsTimer(enable, clk, reset, OneMsTimeOut, Count_out);
    input enable, clk, reset;
    output reg OneMsTimeOut;
    output reg [15:0] Count_out;
    //output [2:0] Count_out; // for sim only
    //reg OneMsTimeOut;
    //reg [15:0] Count_out;
    //reg [2:0] Count_out; // for sim only

    always @(posedge clk) begin
        if (reset == 1'b0) begin
            Count_out <= 16'd0;
            OneMsTimeOut <= 1'b0;
        end
        else begin
            if (enable == 1'b1) begin
                if (Count_out == 16'd50000) begin
                    Count_out <= 16'd0;
                    OneMsTimeOut <= 1'b1;
                end
                else begin
                    Count_out <= Count_out+1;
                    OneMsTimeOut <= 1'b0;
                end
            end
        end
    end
end

endmodule

```

```
// ECE 6370
// Author: Tru Nguyen
// Module name: OneSecTimer
// Function: the top-level timing module that generates a 1-
second timeout signal by cascading three smaller timers.
// It uses a 1 ms timer, then counts those pulses to form 100
ms, and finally counts again to produce a 1-second pulse.
```

```
module OneSecTimer(enable, clk, reset, OneSecTimeOut);
    input enable, clk, reset;
    output OneSecTimeOut;
    wire OneMsTimeOut, HundredMsTimeOut;

    OneMsTimer One_Ms_Timer(enable, clk, reset, OneMsTimeOut);

    countTo100 count_To_100(OneMsTimeOut, clk, reset,
HundredMsTimeOut);

    countTo10 count_To_10(HundredMsTimeOut, clk, reset,
OneSecTimeOut);
```

```
endmodule
```

```
module RNG(RNG_Gen, clk, reset, Rand_Numb);
    input RNG_Gen, clk, reset;
    output [3:0] Rand_Numb;

    wire Count;
    wire [3:0] Count_out;

    Counter Counter_top(Count, clk, reset, Count_out);

    assign Count = ~RNG_Gen;
    assign Rand_Numb = Count_out;

endmodule
```



```

module TwoDigitTimer(reconfig, timer_enable, clk, reset,
timeout, ones_digit_out, tens_digit_out);
    input reconfig, timer_enable, clk, reset;
    output [3:0] ones_digit_out, tens_digit_out;
    output timeout;

    wire BorrowUp_OnesToTens, BorrowUp_Tens;
    wire NoBorrowDN_Ones, NoBorrowDN_Tens; // BorrowUp_Tens is
left floating; NoBorrowUp_Tens is wired to logic high 1'b1
    wire OneSecTimeOut;

    digitTimer Ones_Digit(reconfig, OneSecTimeOut,
NoBorrowDN_Tens, clk, reset, ones_digit_out,
BorrowUp_OnesToTens, NoBorrowDN_Ones);

    digitTimer Tens_Digit(reconfig, BorrowUp_OnesToTens, 1'b1,
clk, reset, tens_digit_out, BorrowUp_Tens, NoBorrowDN_Tens);

    OneSecTimer One_Sec_Timer(timer_enable, clk, reset,
OneSecTimeOut);

    assign timeout = NoBorrowDN_Ones;
endmodule

```

```

// Author: Tru Nguyen
// Module name: testbench_OneSecTimer
// Function: A simulation testbench that verifies the
functionality of the OneSecTimer module by generating a clock
signal and applying reset and enable stimuli to observe the
OneSecTimeOut output.

`timescale 1 ns/100 ps

module testbench_OneSecTimer();
    reg clk, reset, enable;
    wire OneSecTimeOut;

    always
    begin
        clk = 1'b0;
        #10;
        clk = 1'b1;
        #10;
    end

    OneSecTimer DUT_OneSecTimer(enable, clk, reset,
OneSecTimeOut);

    initial
    begin
        reset = 1'b1; // not pressed
        enable = 1'b0;

        @(posedge clk);
        @(posedge clk);
        #5 reset = 1'b0; // reset is pressed
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        #5 reset = 1'b1; // released
        @(posedge clk);
        @(posedge clk);
        #5 enable = 1'b1; // start counting
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);

    end

endmodule

```

```

`timescale 1 ns/100 ps

module testbench_TwoDigitTimer();
    reg reconfig, timer_enable, clk, reset;
    wire timeout;
    wire [3:0] ones_digit_out, tens_digit_out;

    always
    begin
        clk = 1'b0;
        #10;
        clk = 1'b1;
        #10;
    end

    TwoDigitTimer DUT_TwoDigitTimer(reconfig, timer_enable, clk,
reset, timeout, ones_digit_out, tens_digit_out);

    initial
    begin
        reset = 1'b1; // not pressed
        timer_enable = 1'b0;
        reconfig = 1'b0;

        @(posedge clk);
        @(posedge clk);
        #5 reset = 1'b0; // reset is pressed
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        #5 reset = 1'b1; // released
        @(posedge clk);
        @(posedge clk);
        #5 reconfig = 1'b1;
        @(posedge clk);
        #5 reconfig = 1'b0;
        @(posedge clk);
        @(posedge clk);
        #5 timer_enable = 1'b1;

    end
endmodule

```

```

`timescale 1 ns/100 ps
module testbench_RNG();
    reg RNG_Gen, clk, reset;

    wire [3:0] Rand_Numb;

    always
    begin
        clk = 1'b0;
        #10;
        clk = 1'b1;
        #10;
    end

    RNG DUT_RNG(RNG_Gen, clk, reset, Rand_Numb);

    initial
    begin
        reset = 1'b1; // not pressed
        @(posedge clk);
        @(posedge clk);
        #5 reset = 1'b0;
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        #5 reset = 1'b1;
        @(posedge clk);
        @(posedge clk);
        RNG_Gen = 1'b0;
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        RNG_Gen = 1'b1;
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        RNG_Gen = 1'b0;
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        RNG_Gen = 1'b1;
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        RNG_Gen = 1'b0;
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        @(posedge clk);
        RNG_Gen = 1'b1;
    end
end

```